The Paving as a Representation of Nondominated Points for the Bi-objective Uncapacitated Facility Location Problem

RAMOO'2026

10-11 September 2026, Istanbul – Türkiye

Xavier GANDIBLEUX

in collaboration with Anthony PRZYBYLSKI

Nantes Université – France

Timeline of this work

2012: a first version (C++)

S. Bourougaa, A. Derrien, A. Grimault, X. Gandibleux and A. Przybylski. Sur le calcul des solutions efficaces du problème bi-objectif de localisation de services sans contrainte de capacité. ROADEF'2012. 11-13 Avril 2012, Angers, France.

X. Gandibleux, A. Przybylski , S. Bourougaa, A. Derrien, A. Grimault. Computing the Efficient Frontier for the 0/1 Biobjective Uncapacitated Facility Location Problem. CORS/MOPGP'2012. June 11-13, 2012, Niagara Falls, Canada.

2022: a new improved version (Julia)

A. Przybylski, X. Gandibleux. Recent multi-objective branch and bound algorithms and an example in facility location. EWGLA XXVII: EURO Working Group on Locational Analysis. September 14-16, 2022. University of Aveiro, Portugal.

2026: a cutting-edge version (Julia, on Github)

X. Gandibleux, A. Przybylski. Paving and computing the set of nondominated points for the bi-objective 0/1 uncapacitated facility location problem. Optimization-Online: `https://optimization-online.org/?p=34431` (2026/04/15).

0/1 biobjective uncapacitated facility location problem

$$
\left[\begin{array}{lll}
\min & f^k(x,s) = \sum_{i\in I}\sum_{j\in J} c_{ij}^k x_{ij} + \sum_{j\in J} r_j^k s_j & k = 1,2 \quad (0) \\
s/c & \sum_{j\in J} x_{ij} = 1 & \forall i \in I \quad (1) \\
 & x_{ij} \leqslant s_j & \forall i \in I,\ \forall j \in J \quad (2) \\
 & x_{ij},\ s_j \in \{0,1\} & \forall i \in I,\ \forall j \in J \quad (3)
\end{array}\right] \quad 2 - UFLP
$$

- single objective version
  - NP-hard [Krarup and Pruzan, 1983]
  - practical situations (example in telecom [Gourdin and al. 2002])
- multi-objective version
  - exact and approximated algorithms for small size bi (multi) objective instances [Fernandez and Puerto 2003]
  - data and numerical instances available [Fernandez and Puerto 2003; Harris and al. 2009; 2011]

A algorithm in three phases using boxes and paving

Box, paving, and remarkable points (1/2)

Let $J_1 \subseteq J$, set of indexes for facilities opened

↳ y^R: point/running costs

Box $\mathcal{B}$:

- Sub-set of dimension 2 in Y bounded by (at most) 2 points corresponding to lexicographic optimal solutions on both objectives for J_1

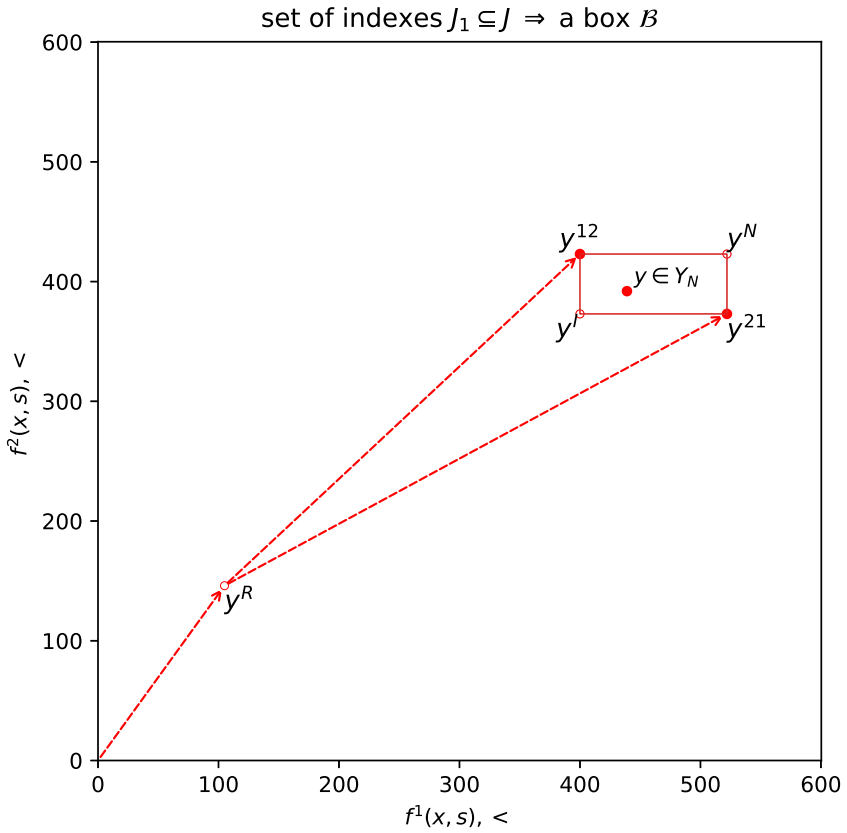
- Remarkable points:

  y^{12}, y^{21}: lex-optimal points

  ↳ y^I: ideal point

  ↳ y^N: nadir point

  $\#J_1 = 1 \Rightarrow$ a same single point



Paving $\mathcal{P}$:

- a collection of boxes such that $Y_N \subset \bigcup_{\mathcal{B} \in \mathcal{P}} \mathcal{B}$

Box, paving, and remarkable points (2/2)

Given $J_1 \subseteq J$, set of indexes for facilities opened

- y^R:
  Straightforward:
$$y^R = \left(\sum_{j \in J_1} r_j^k, \ k = 1, 2 \right)$$

- y^{12}, y^{21}:
  Lexicographic resolution of a single objective allocation problem:

$$y^{A^k} = \left(\begin{array}{lll} \min_{lex} & f^k(x) = \sum_{i \in I} \sum_{j \in J_1} c_{ij}^k x_{ij} & (0) \\ s/c & \sum_{j \in J_1} x_{ij} = 1 \quad \forall i \in I & (1) \\ & x_{ij} = 0 \quad \forall i \in I, \ \forall j \in J \backslash J_1 & (2) \\ & x_{ij} \in \{0, 1\} \quad \forall i \in I, \ \forall j \in J_1 & (3) \end{array} \right), \ k = 1, 2$$

  giving
$$y^{12} = y^R + y^{A^1}$$
$$y^{21} = y^R + y^{A^2}$$

Observation

$$\left(\bigcup_{\mathcal{B} \in \mathcal{P}} \{y^{12}\} \cup \{y^{21}\} \right)_N$$

defines an upper bound set U for Y_N

A 3-phase algorithm to compute Y_N for the 2-UFLP

Algorithm 1 `2-UFLP solver`

input: *data* ▷ numerical instance

output: Y_N ▷ nondominated points

1: *initialPaving* ← **computePaving**(*data*) ▷ phase 1

2: *improvedPaving* ← **improvePaving**(*data*, *initialPaving*) ▷ phase 2

3: Y_N ← **generateNDPoints**(*data*, *improvedPaving*) ▷ phase 3

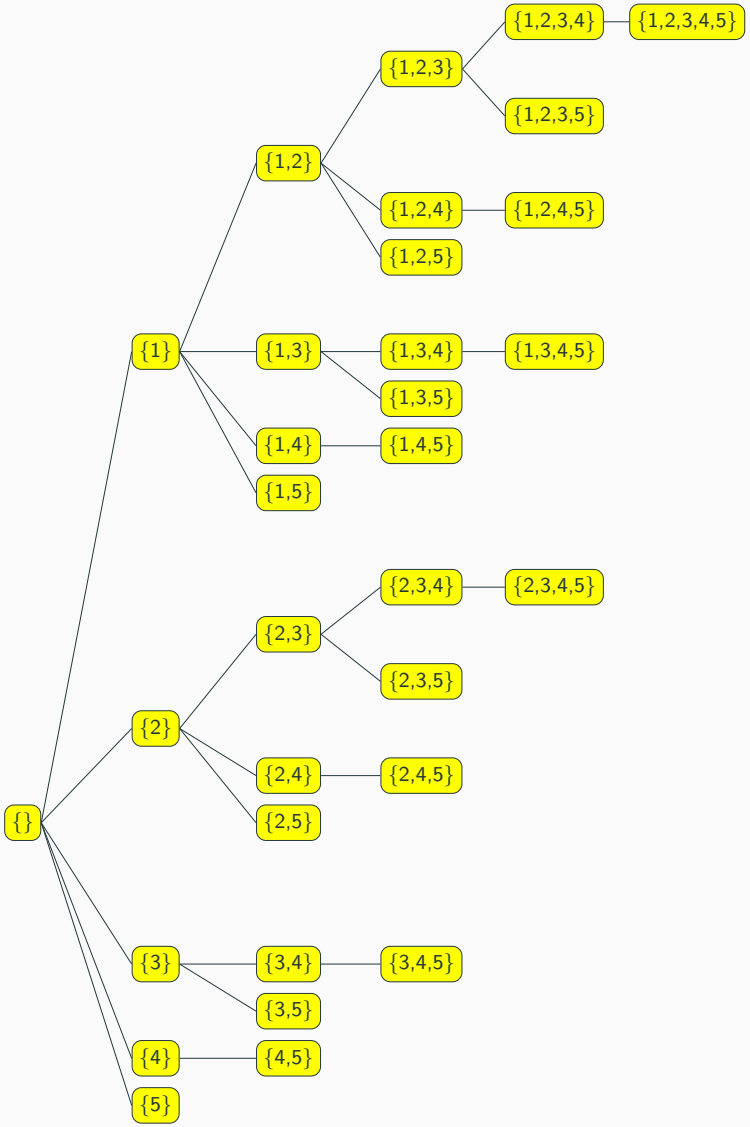
Phase 1: paving Y_N with a branch and bound algorithm

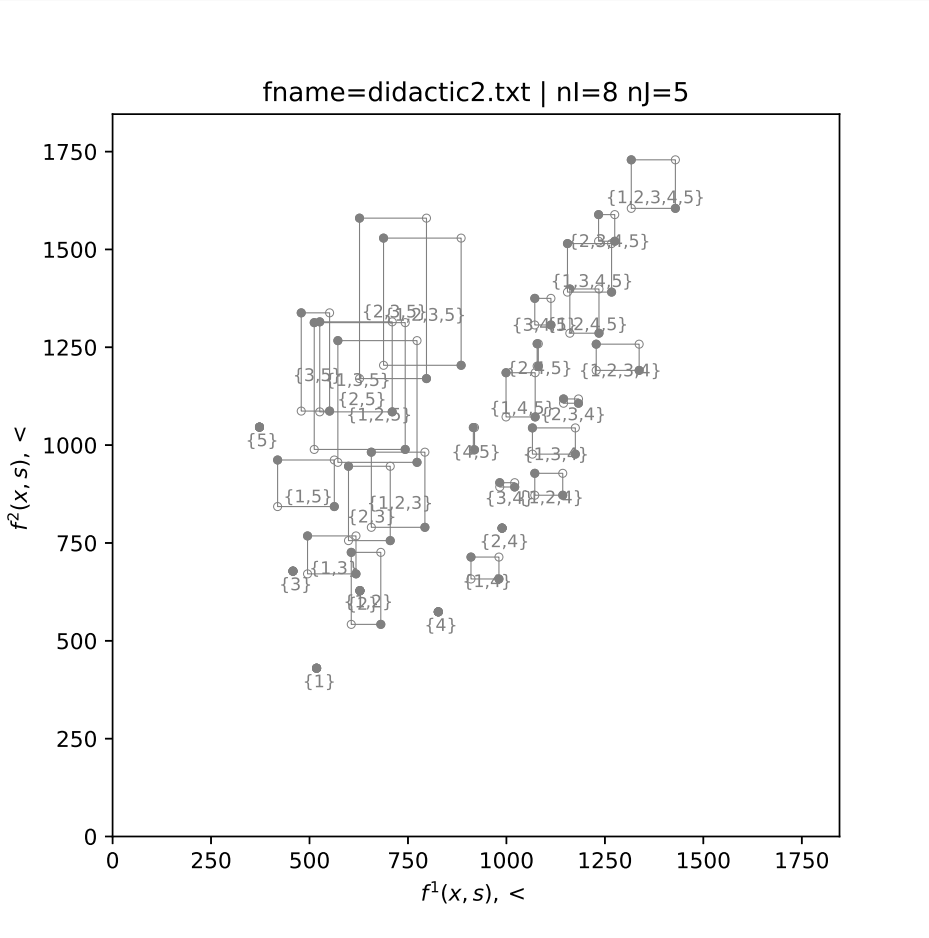
Phase 2: improve the paving with a reduction and filtering algorithm

Phase 3: compute Y_N with a labeling algorithm

PHASE 1: Paving Algorithm

Algorithm: the sets J_1 (example with #J=5)





Building the initial paving: the idea

The search tree is explored **breadth-first**.
Each node = a subset J_1 of opened facilities.
Branching rule = variables s_j

Two roles for **a node**, tracked as **two queues**:

- $\mathcal{L}$ unpruned **unexpanded** nodes
- $\mathcal{P}$ unpruned **expanded** nodes (boxes known: current paving)

Associated to **a box** $\mathcal{B}$:

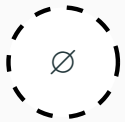
- N a *unexpanded* node
- B a expanded node

Two elementary operations move nodes between queues:

- `branch(`N`)` open one more facility $\Rightarrow$ feeds $\mathcal{L}$
- `expand(`N`)` compute the box of N $\Rightarrow$ feeds $\mathcal{P}$, may update U

Algorithm: principle for building the tree

(a) **Initial state:** creating the root

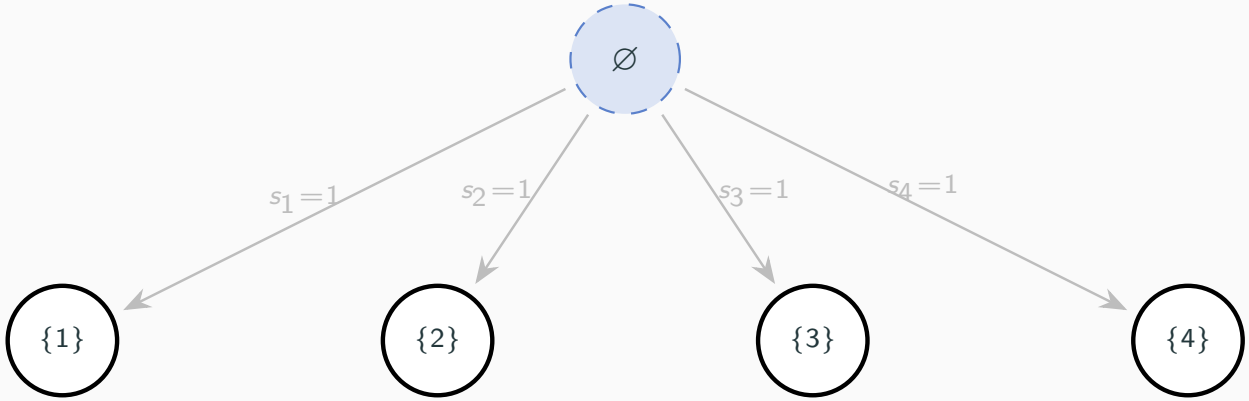


$\mathcal{L} = \varnothing$
$\mathcal{P} = \varnothing$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Algorithm: principle for building the tree

(b) **Root processed:** its 4 sons are added to $\mathcal{L}$

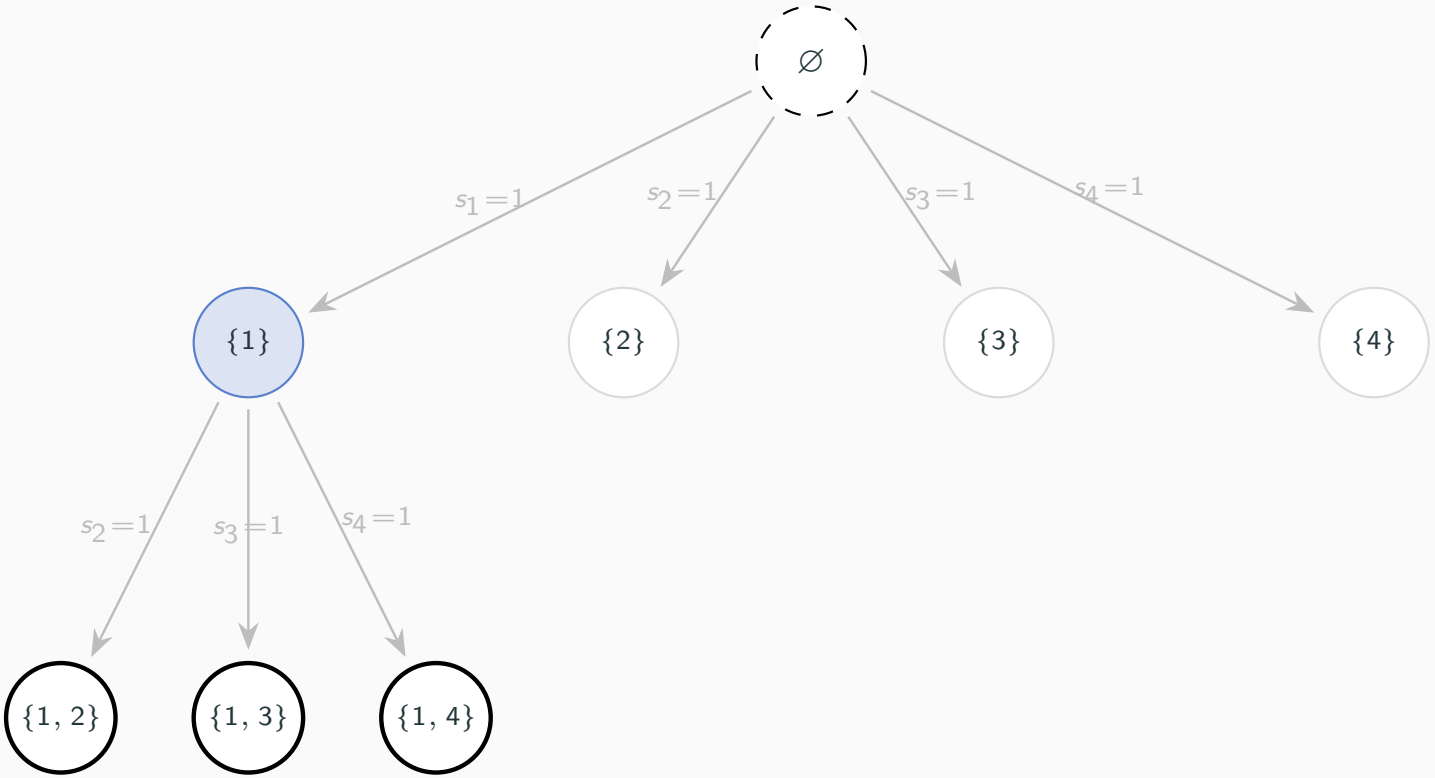


$\mathcal{L}=\{\{1\},\{2\},\{3\},\{4\}\}$
$\mathcal{P}=\emptyset$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Algorithm: principle for building the tree

(c) $J_1 = \{1\}$ **processed:** its 3 sons are added to $\mathcal{L}$

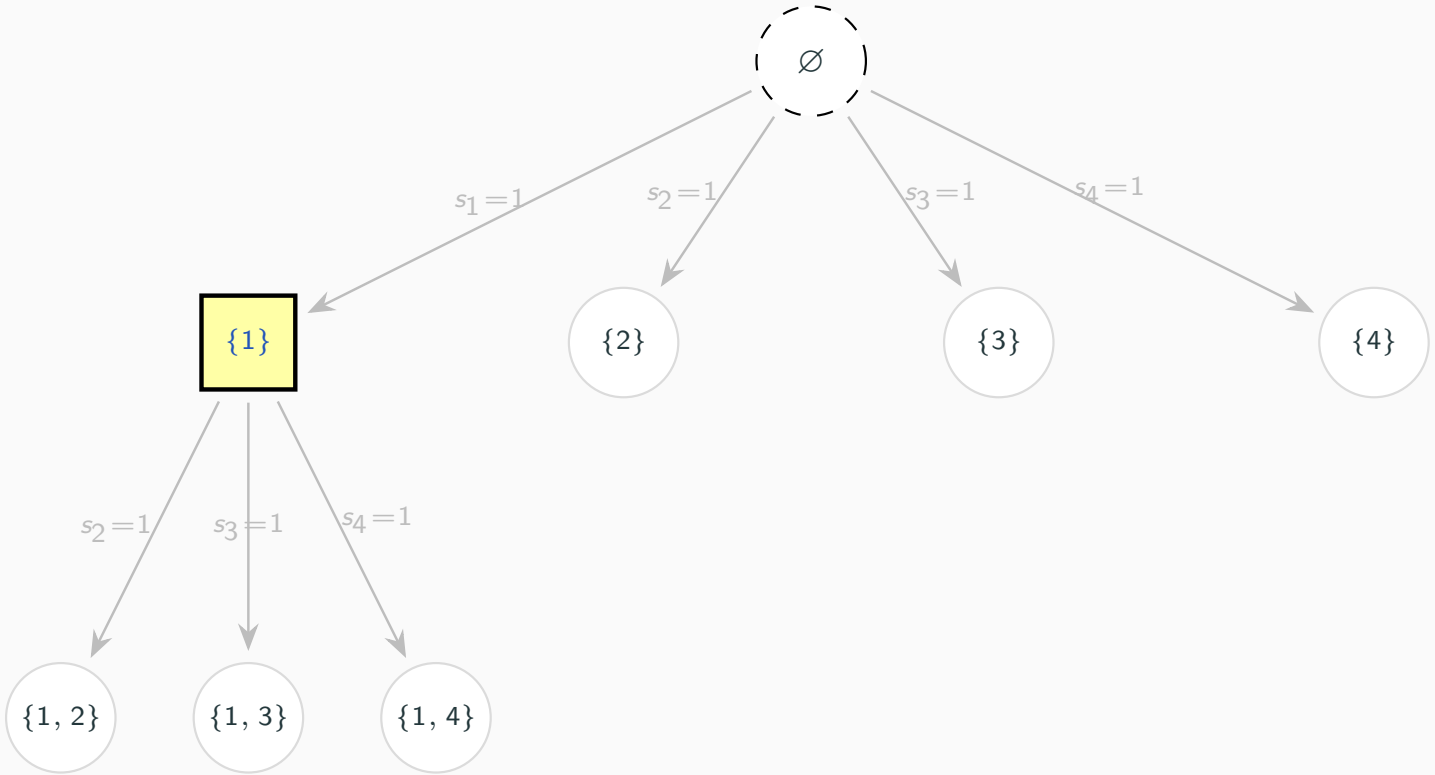


$\mathcal{L} = \{\{1\}, \{2\}, \{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}\}$
$\mathcal{P} = \varnothing$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Algorithm: principle for building the tree

(d) $J_1 = \{1\}$ expanded and added to $\mathcal{P}$

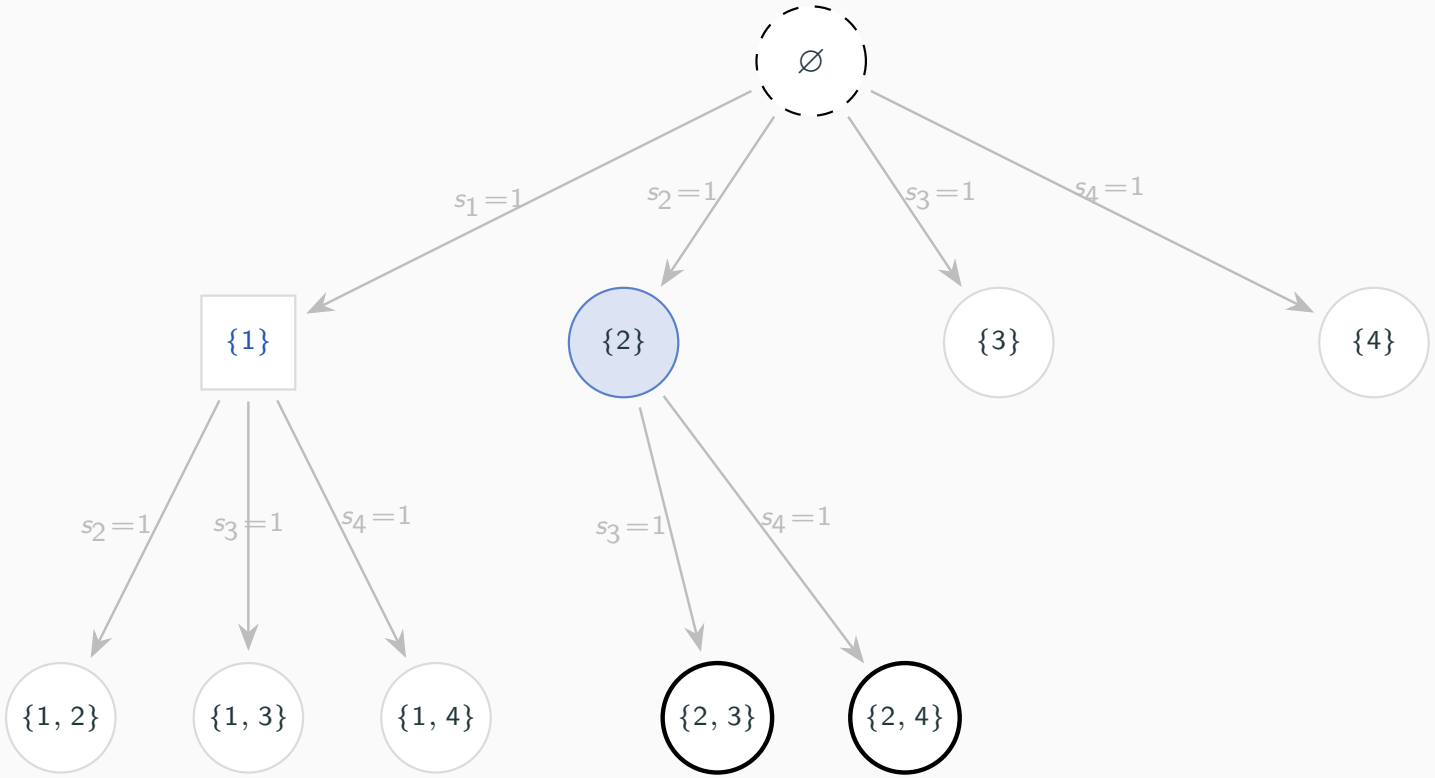


$\mathcal{L} = \{\{2\}, \{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}\}$
$\mathcal{P} = \{\{1\}\}$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Algorithm: principle for building the tree

(e) $J_1 = \{2\}$ **processed:** its 2 sons are added to $\mathcal{L}$

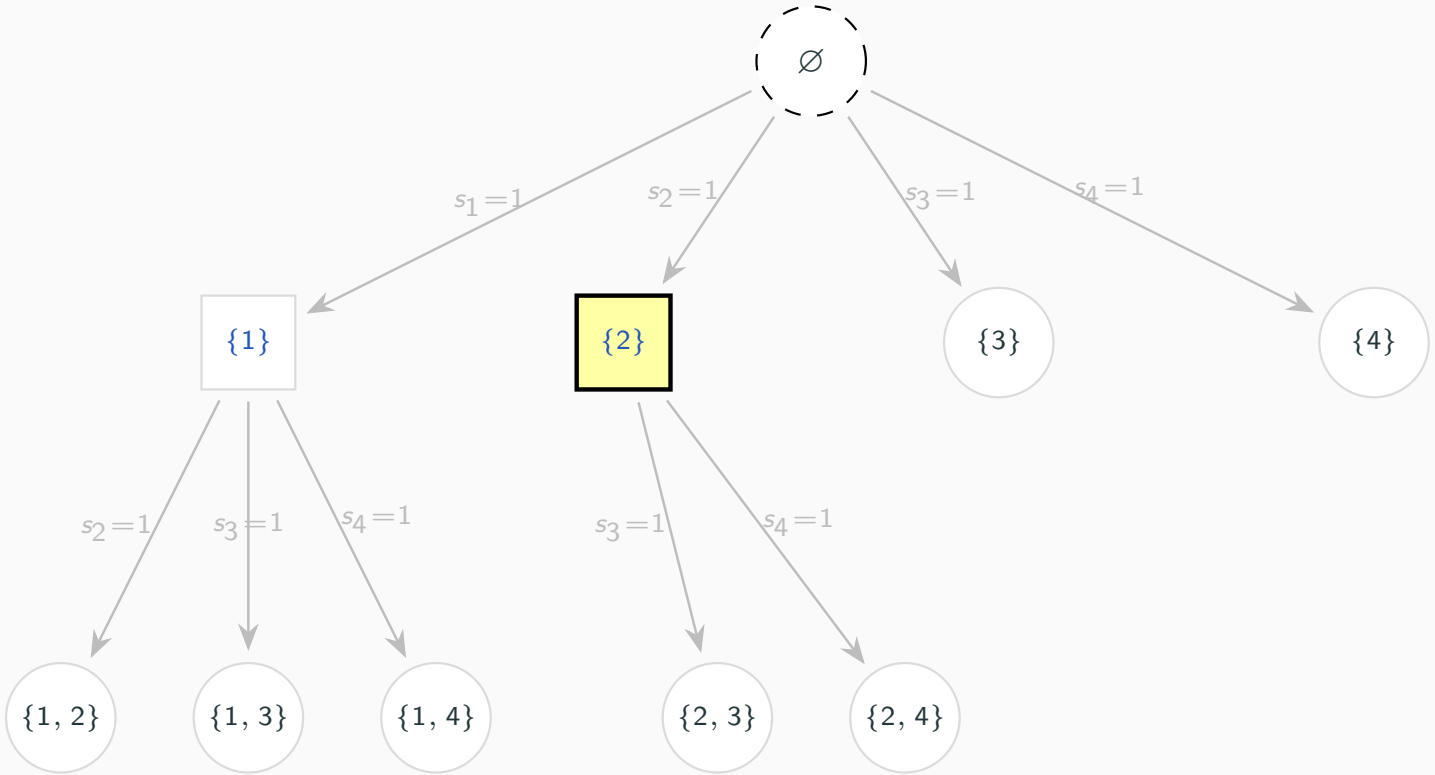


$\mathcal{L} = \{\{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}\}$
$\mathcal{P} = \{\{1\}\}$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Algorithm: principle for building the tree

(f) $J_1 = \{2\}$ **expanded and added to** $\mathcal{P}$; etc.



$\mathcal{L} = \{\{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}\}$
$\mathcal{P} = \{\{1\}, \{2\}\}$

$\mathcal{L}$: unexpanded nodes (circle) – $\mathcal{P}$: expanded nodes (square, current paving)

Three dominance tests decide what survives

N: a new unexpended node related to a given box

B: a new expended node related to a given box

$B' \in \mathcal{P}$: current expended nodes

$u \in U$: a point of the upper bound set U

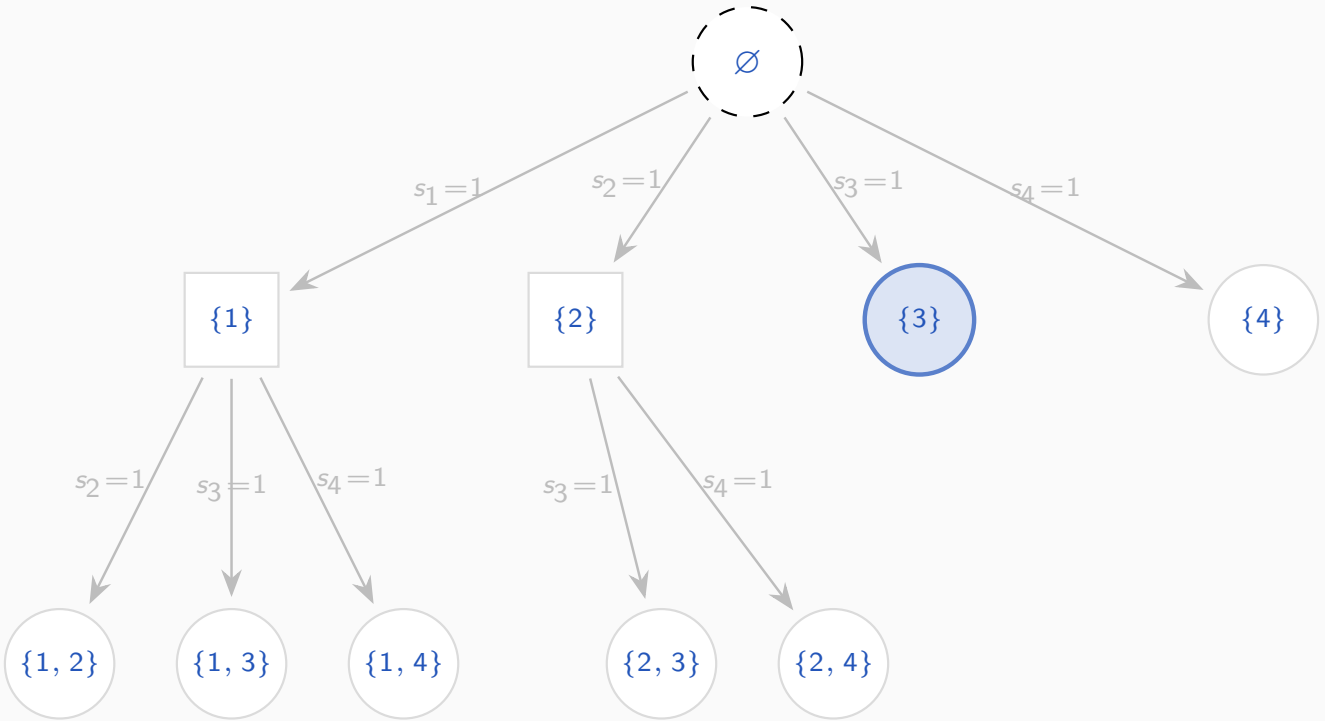
- Test 1:
  check if y^R *of* N is weakly dominated by one $u \in U$
  ↳ yes $\Rightarrow$ N pruned. Stop

- Test 2:
  check if y^I *of* B is weakly dominated by one $u \in U$
  ↳ yes $\Rightarrow$ B pruned. Stop

- Test 3:
  check if y^I *of* B' is weakly dominated by y^{12}, y^{21} *of* B
  ↳ yes $\Rightarrow$ B' pruned. Continue with others $B' \in \mathcal{P}$

Algorithm: principle for filtering nodes (Test 1)

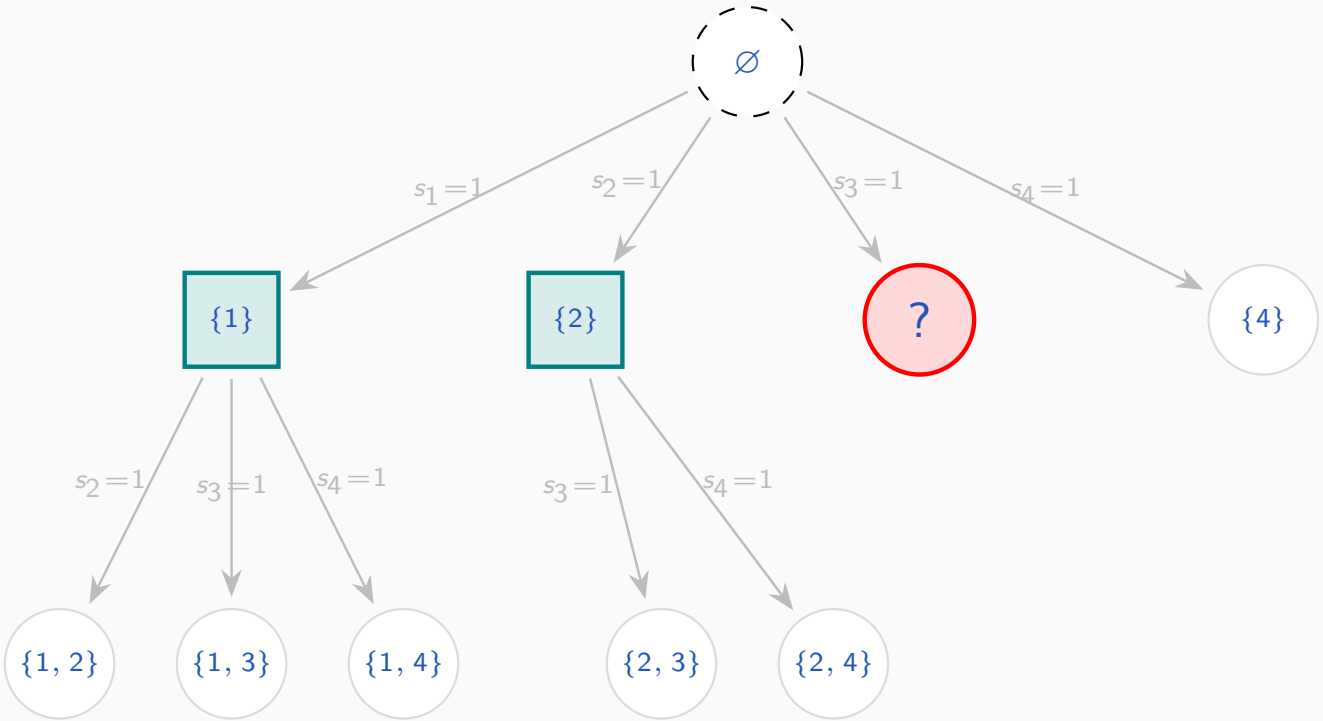
(a) Node $J_1 = \{3\}$ is the next node selected in $\mathcal{L}$



$\mathcal{L} = \{\{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}\}$
$\mathcal{P} = \{\{1\}, \{2\}\}$

Algorithm: principle for filtering nodes (Test 1)

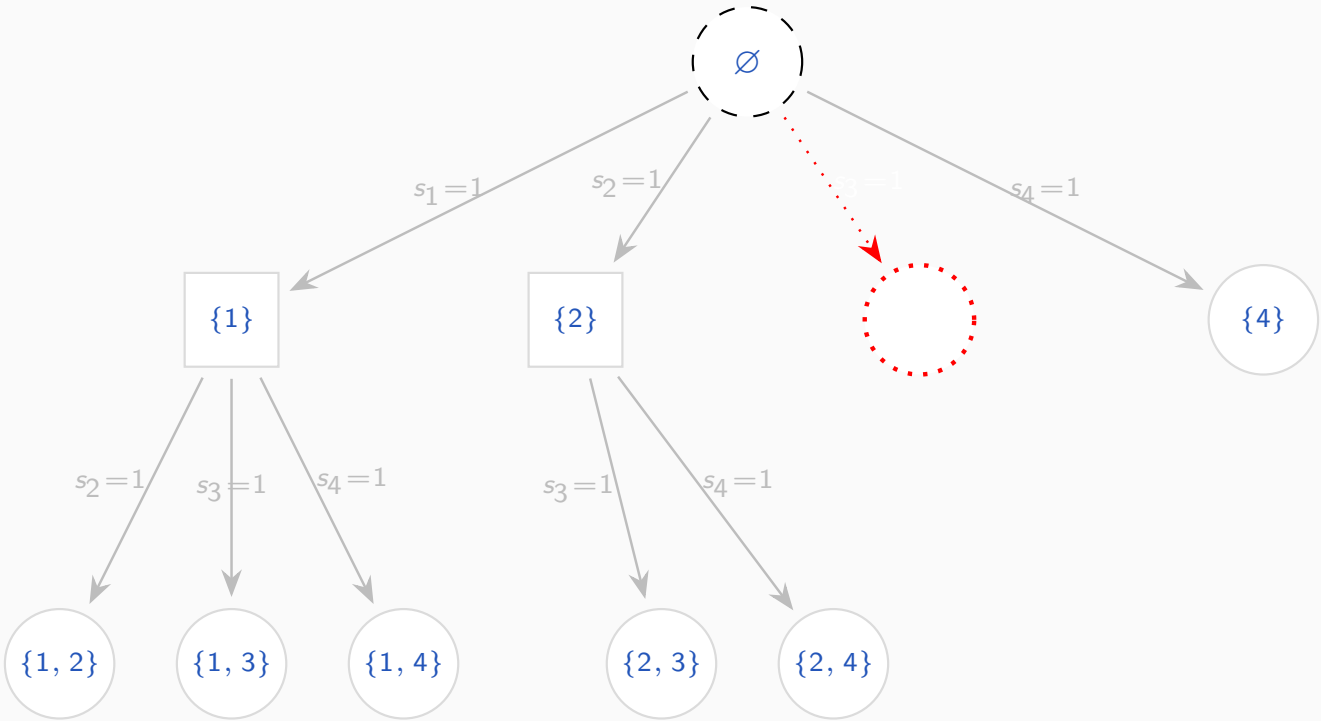
(b) Running `Test1` on node $J_1 = \{3\}$



$\mathcal{L} = \{\{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}\}$
$current = \{3\}$; $\mathcal{P} = \{\{1\}, \{2\}\}$

Algorithm: principle for filtering nodes (Test 1)

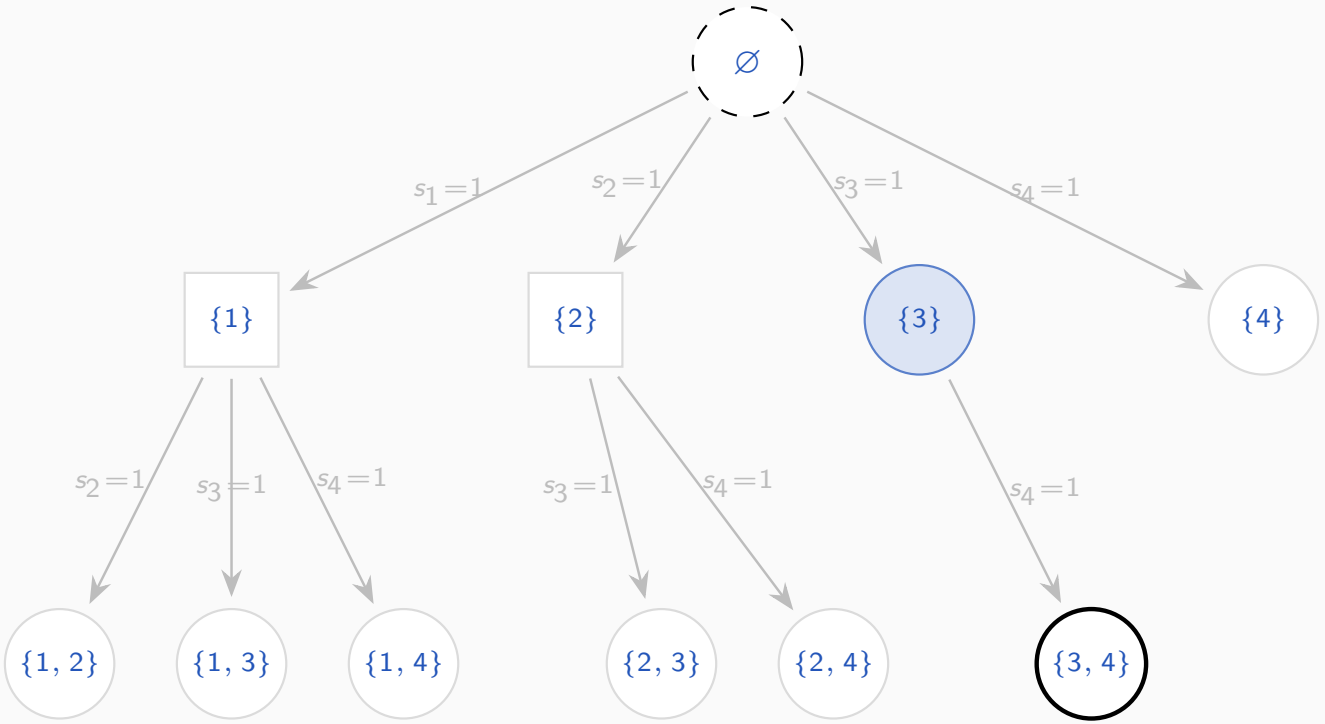
(c) Test1=true $\Rightarrow$ node $J_1=\{3\}$ pruned



$\mathcal{L}=\{\{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\}, \{2\}\}$

Algorithm: principle for filtering nodes (Test 1)

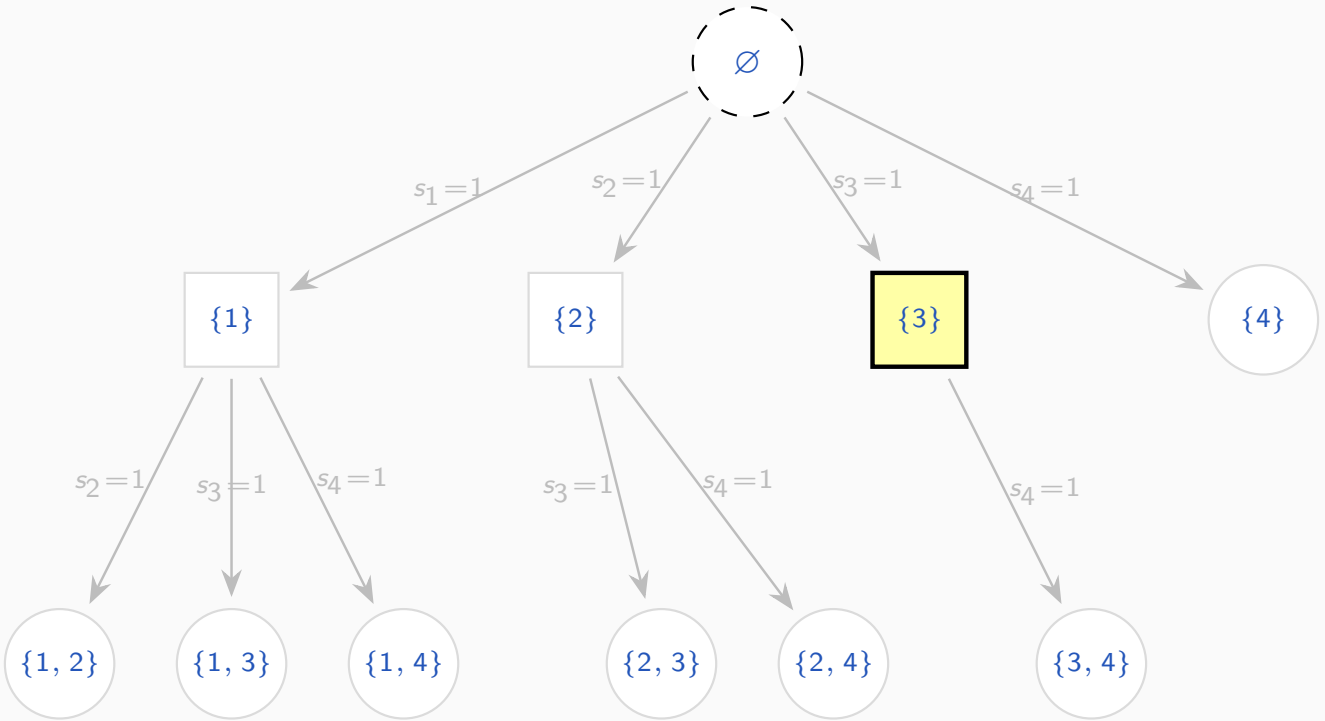
(d) Test1=false $\Rightarrow$ Branch: node $J_1=\{3,4\}$ added to $\mathcal{L}$



$\mathcal{L}=\{\{4\},\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\},\{2\}\}$

Algorithm: principle for filtering nodes (Test 1)

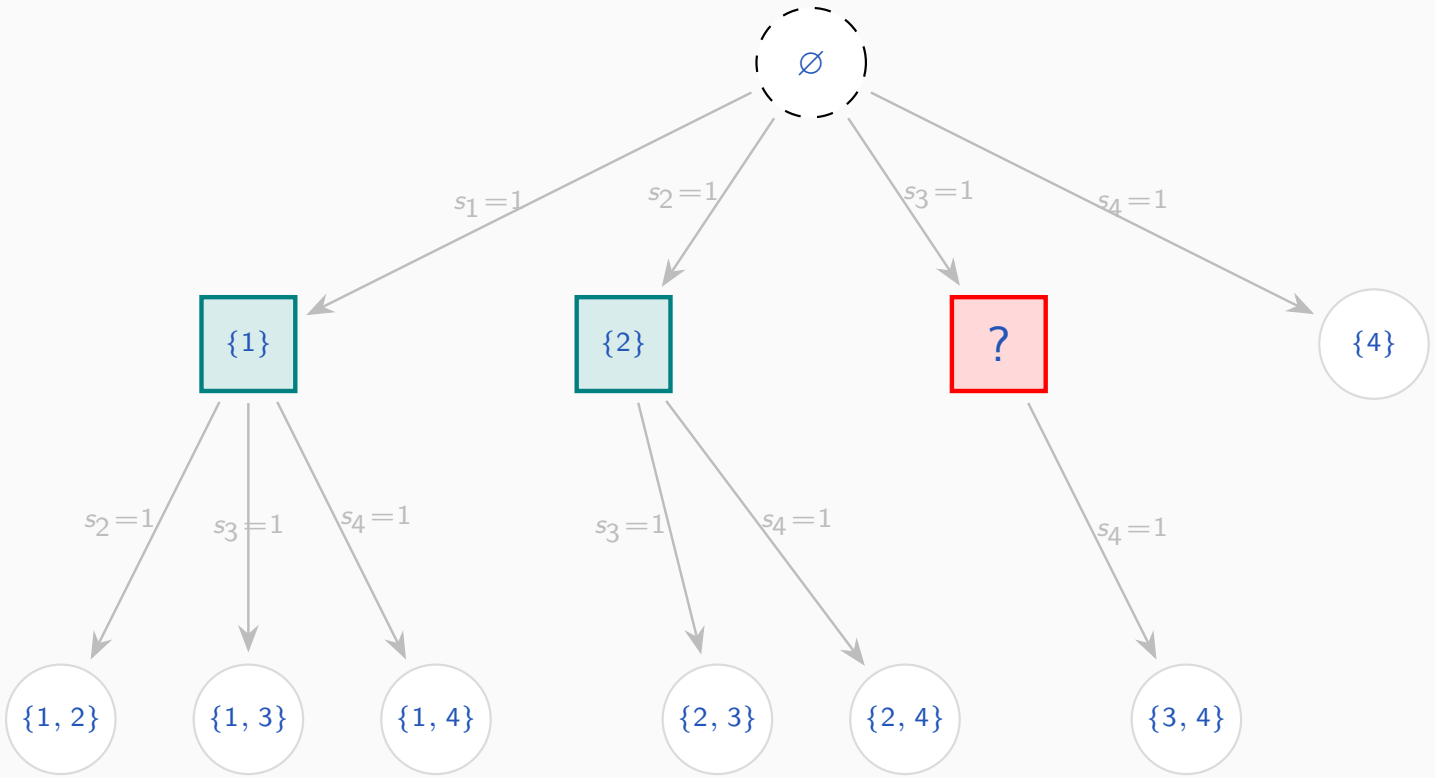
(e) Test1=false $\Rightarrow$ Expand: box $J_1=\{3\}$ added to $\mathcal{P}$



$\mathcal{L}=\{\{4\},\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\},\{2\},\{3\}\}$

Algorithm: principle for filtering boxes (Test 2 and 3)

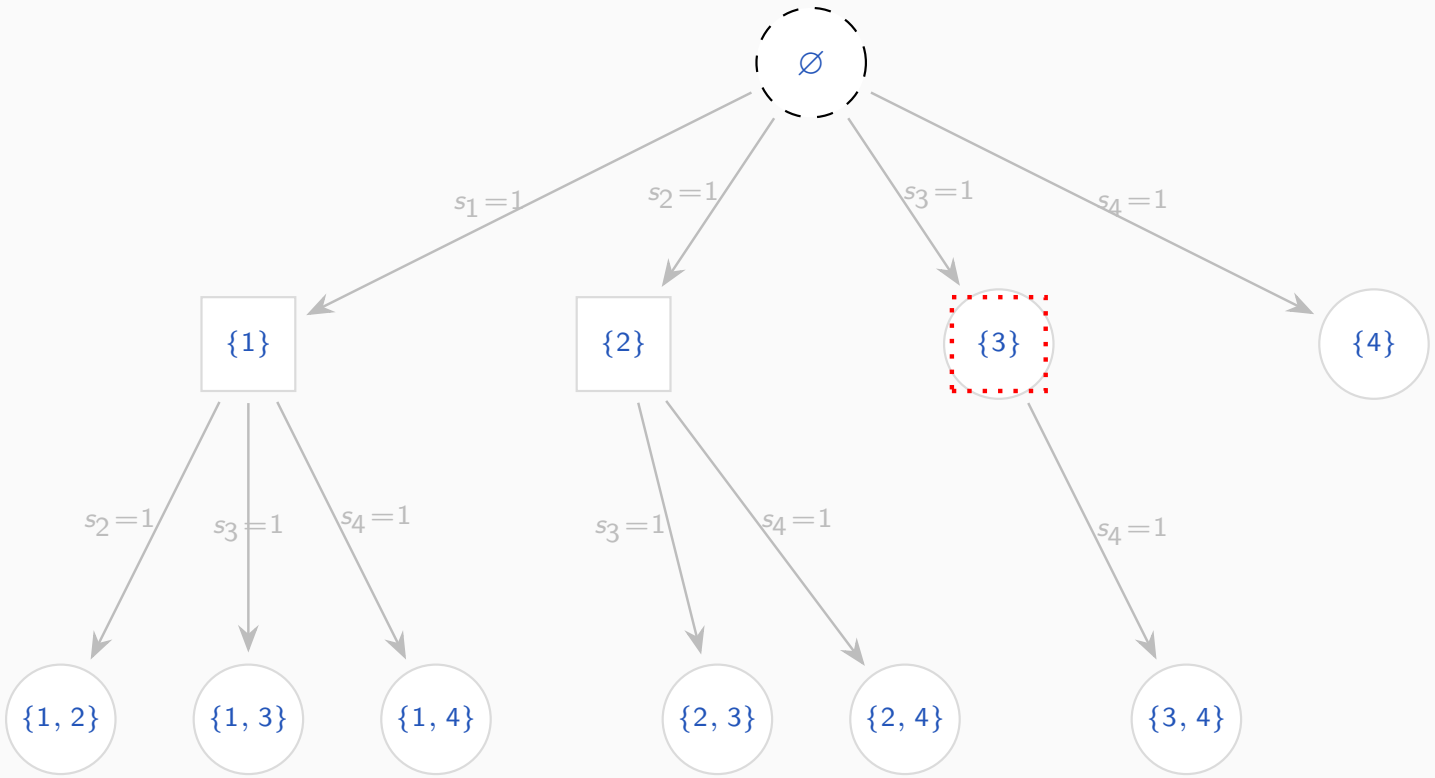
(a) Running `Test2` on box $J_1 = \{3\}$



$\mathcal{L} = \{\{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}, \{3,4\}\}$
$current = \{3\}$; $\mathcal{P} = \{\{1\}, \{2\}, \{3\}\}$

Algorithm: principle for filtering boxes (Test 2 and 3)

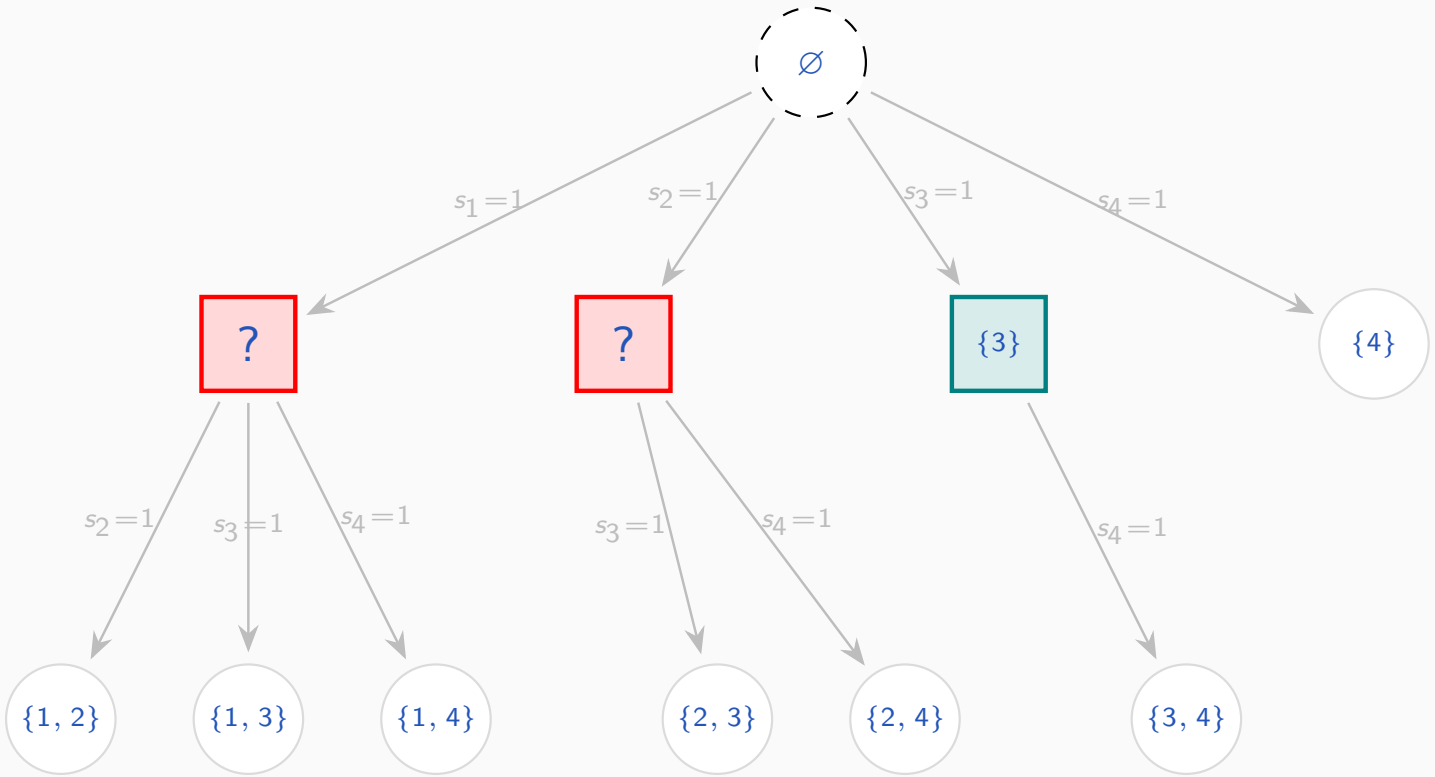
(b) Test2=true $\Rightarrow$ box $J_1=\{3\}$ pruned



$\mathcal{L}=\{\{4\},\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\},\{2\}\}$

Algorithm: principle for filtering boxes (Test 2 and 3)

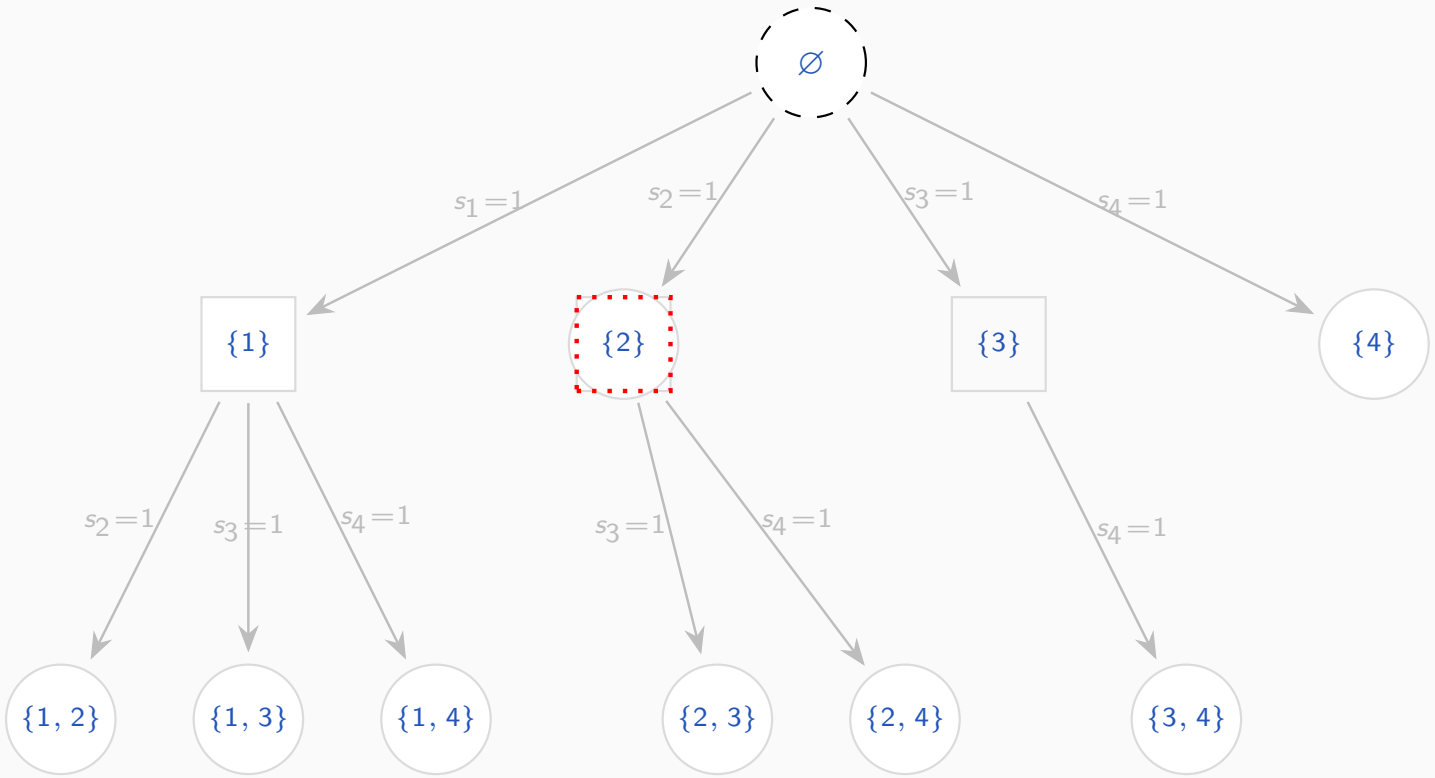
(c) `Test2`=false ⇒ Running `Test3` using new feasible points associated to $J_1=\{3\}$



$\mathcal{L}=\{\{4\},\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\},\{2\},\{3\}\}$

Algorithm: principle for filtering boxes (Test 2 and 3)

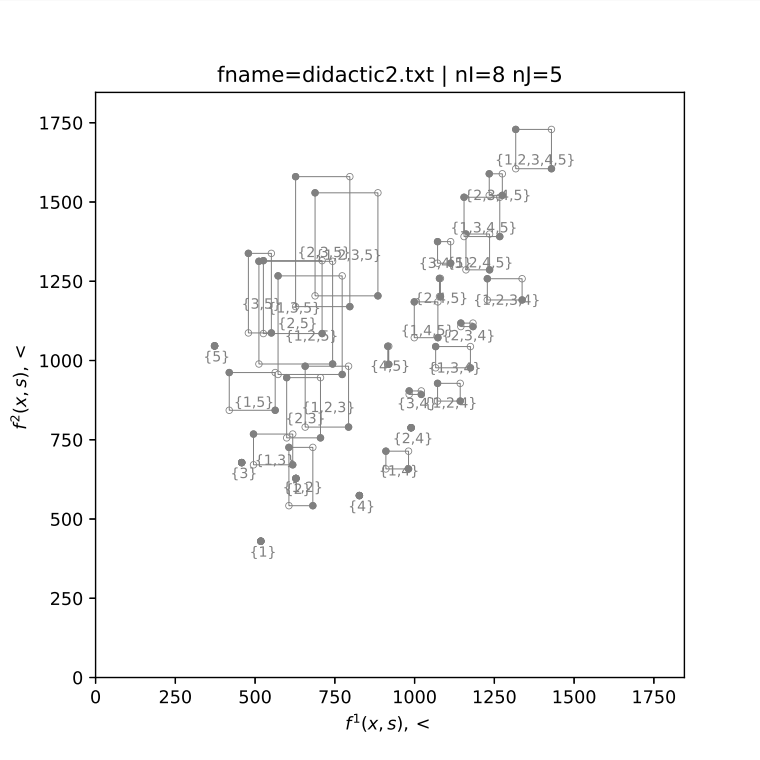
(d) Test3=true $\Rightarrow$ box $J_1=\{2\}$ pruned



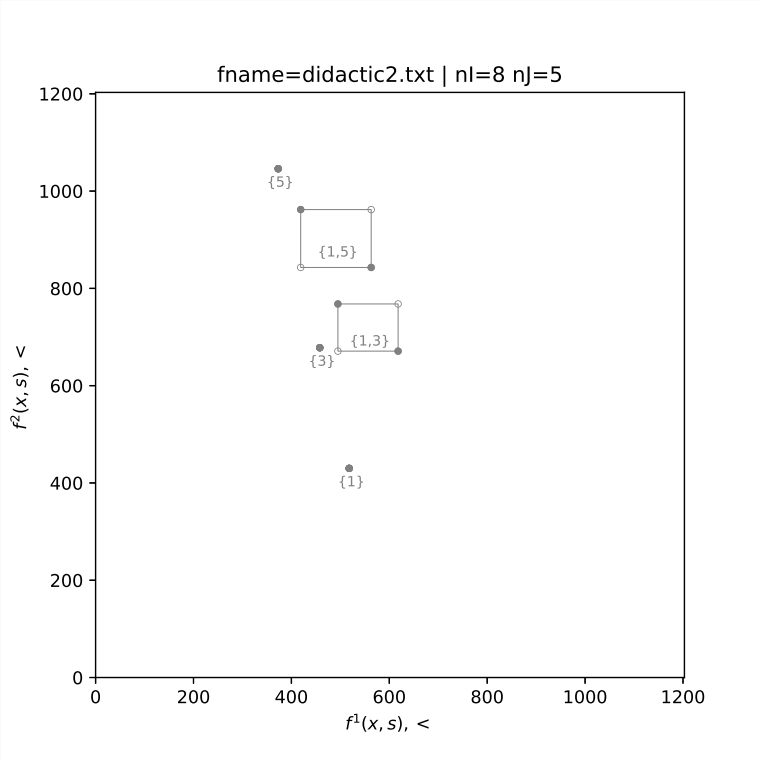
$\mathcal{L}=\{\{4\},\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}$
$current=\{3\}$; $\mathcal{P}=\{\{1\},\{3\}\}$

Didactic example (with #J=5)

Paving obtained :



without pruning tests



with pruning tests

PHASE 2: Refining Algorithm

Improve Paving algorithm

Algorithm 2 `improvePaving`

input: $\mathcal{P}$, a paving
output: $\mathcal{P}$, an improved paving
`pruned`$[\mathcal{B}] \leftarrow$ `false`, $\forall \mathcal{B} \in \mathcal{P}$
for all $\mathcal{B} \in \mathcal{P}$ **do**
    $S^{\mathcal{B}} \leftarrow$ **computeSuppPoints**$(\mathcal{B}) \cup \{y^{12}, y^{21}\}$ ▷ some Supp. pts
    `sort`$(S^{\mathcal{B}} \updownarrow)$ in ascending order of f_1
end for
for all $(\mathcal{B}_0, \mathcal{B}_1) \in \mathcal{P}$, $\mathcal{B}_0 \neq \mathcal{B}_1$, $\neg$`pruned`$[\mathcal{B}_0]$, $\neg$`pruned`$[\mathcal{B}_1]$ **do**
    tryToShrinkPruneBox$(\mathcal{B}_0 \downarrow, \mathcal{B}_1 \updownarrow,$ **pruned** $\updownarrow)$
end for
`deleteBoxes`(`pruned`$\downarrow$, $\mathcal{P} \updownarrow$) ▷ delete pruned boxes

computeSuppPoints($\mathcal{B}$)

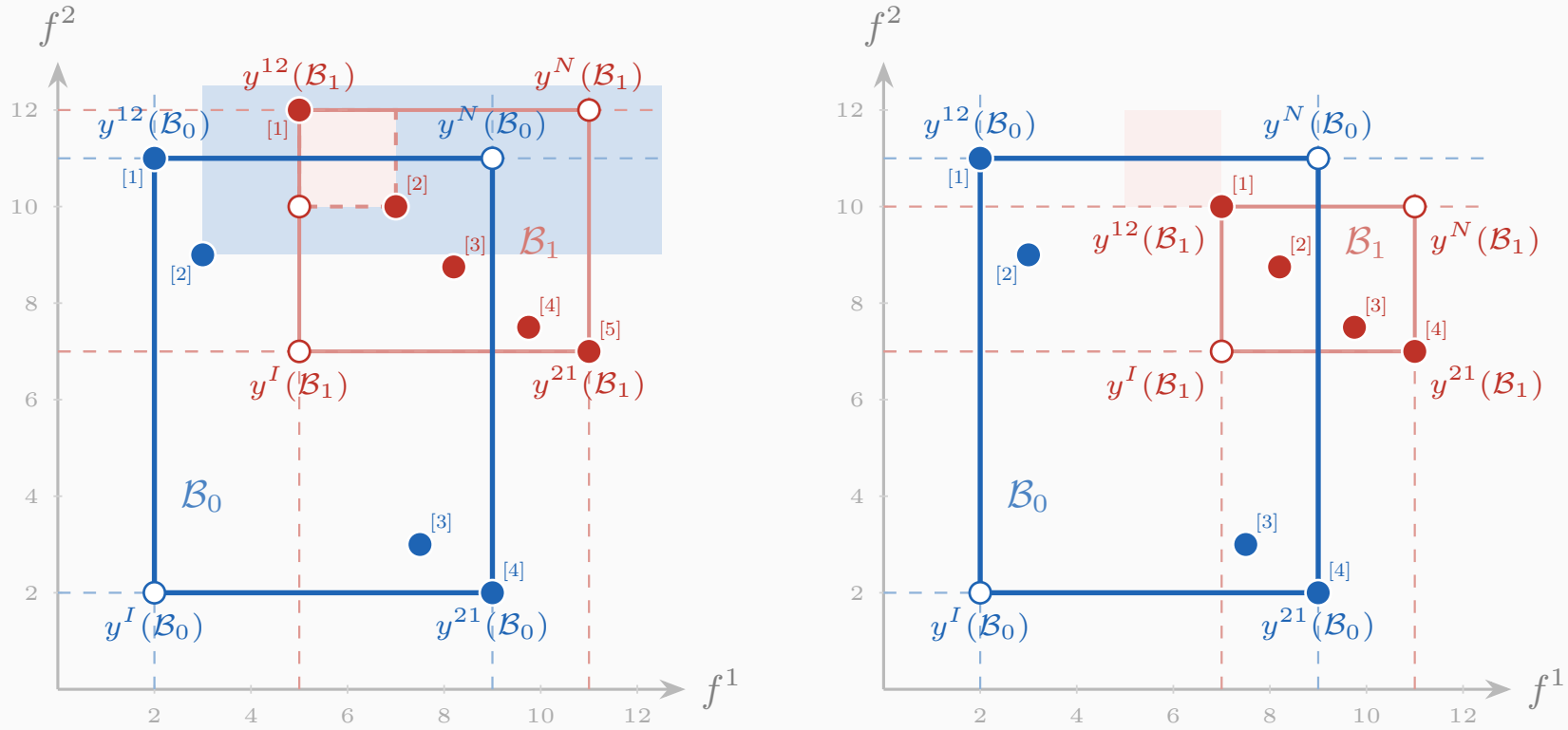
$y \in Y_{NS}$ for $\mathcal{B}$ is obtained, given a weight $\lambda = \frac{\lambda_1}{\lambda_1+\lambda_2} \in [0,1]$, in solving the scalarized bi-objective allocation problem

$$\left[\begin{array}{lll} \min(\lambda) & \sum_{i\in I}\sum_{j\in J_1} \lambda c_{ij}^1 x_{ij} + (1-\lambda) c_{ij}^2 x_{ij} & (0) \\ s/c & \sum_{j\in J_1} x_{ij} = 1 \quad \forall i \in I & (1) \\ & x_{ij} \in \{0,1\} \quad \forall i \in I,\ \forall j \in J_1 & (2) \end{array}\right]$$

which is straightforward

tryToShrinkPruneBox($\mathcal{B}_0 \downarrow, \mathcal{B}_1 \updownarrow$, pruned $\updownarrow$)

Two given boxes: $\mathcal{B}_0$ (the reference), $\mathcal{B}_1$ (the candidate)



Step 1: apply the operator North-West if $y_1^{12}(\mathcal{B}_1) \geqslant y_1^{12}(\mathcal{B}_0)$:

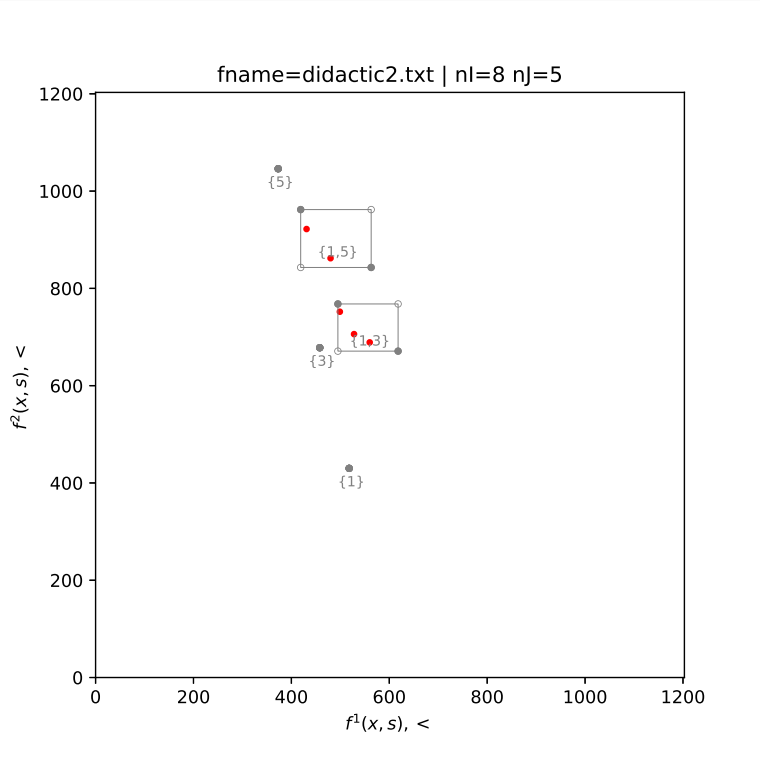
1. Try to shrink $\mathcal{B}_1$ from the North-West, using points of $S^{\mathcal{B}_0}$
2. Try to prune $\mathcal{B}_1$ using a point of $S^{\mathcal{B}_0}$; if successful, stop

Step 2: apply the operator South-East if $y_2^{21}(\mathcal{B}_1) \geqslant y_2^{21}(\mathcal{B}_0)$:

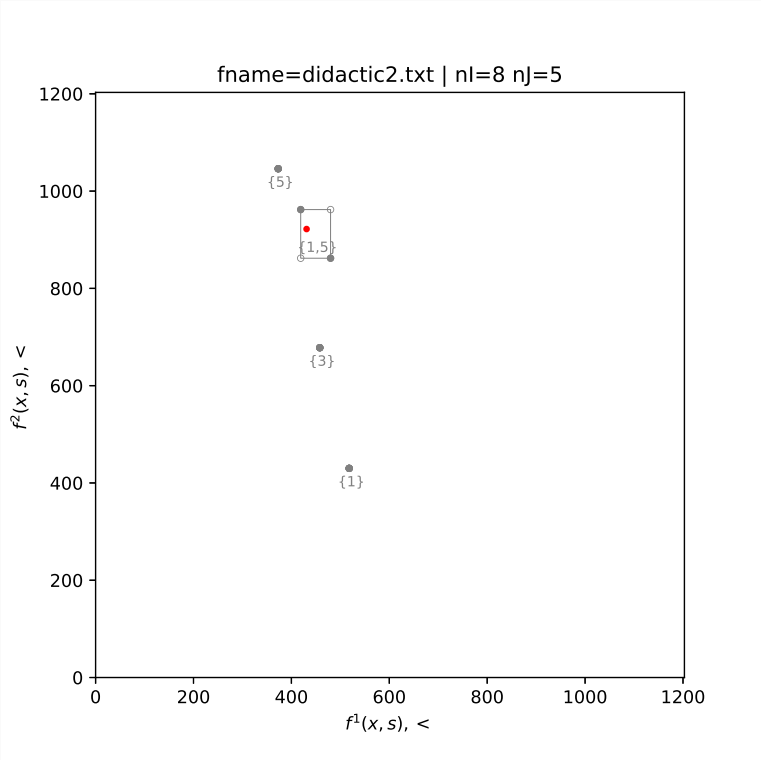
1. Try to shrink $\mathcal{B}_1$ from the South-East, using points of $S^{\mathcal{B}_0}$
2. Try to prune $\mathcal{B}_1$ using a point of $S^{\mathcal{B}_0}$; if successful, stop

Didactic example (with #J=5)

Final paving obtained:



without refining



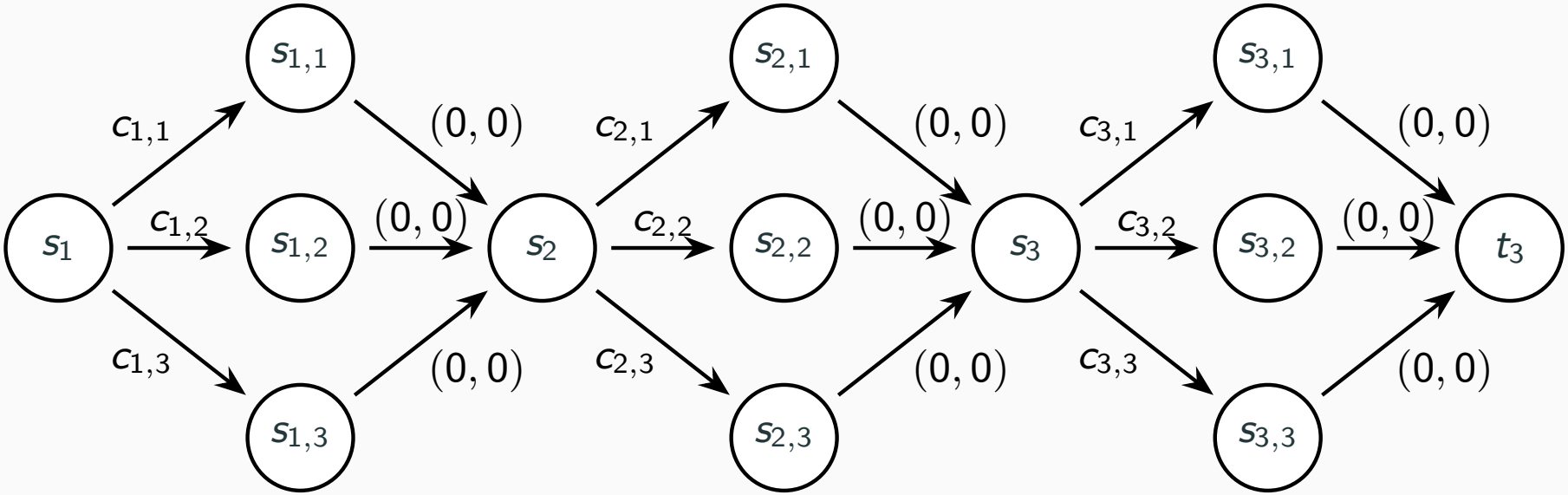
with refining

PHASE 3: Labeling Algorithm

Few words...

- A labeling algorithm to solve bi-objective allocation problems
- Close to Fernández-Puerto (2003) and by Stiglmayr et al. (2014)
- Problem reformulated as a shortest path problem.

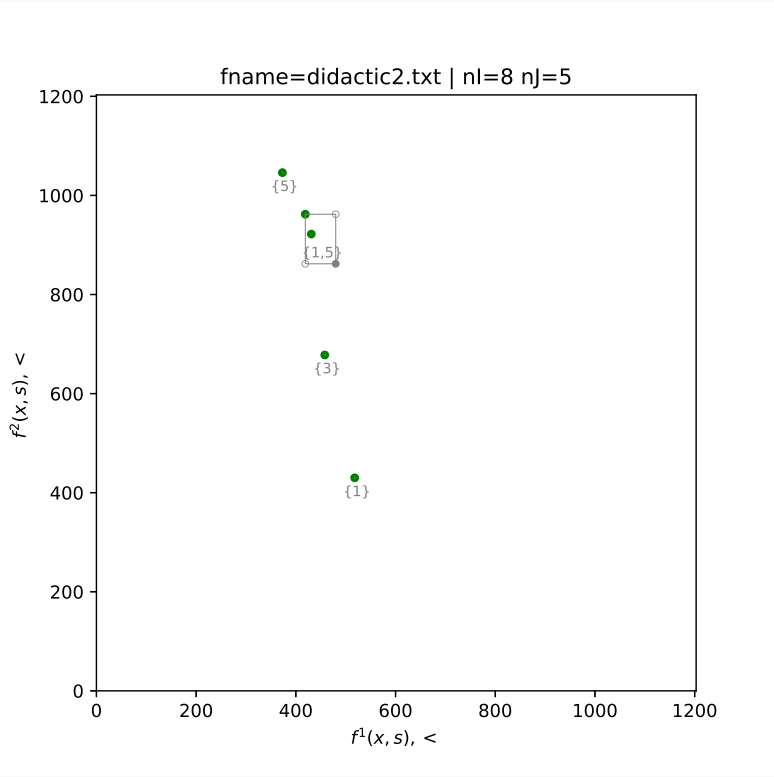
Example of the shortest path formulation of a bi-objective allocation problem with 3 customers and 3 facilities:



Didactic example (with #J=5)

$\mathbb{R}^2$,

the objective space:



Y_N,

the set of nondominated points

X_{E_m},

a minimum set of efficient solutions

costs (f^1, f^2)	services opened	allocation user-service 1, 2, 3, 4, 5, 6, 7, 8
(518, 430)	1	1, 1, 1, 1, 1, 1, 1, 1
(458, 678)	3	3, 3, 3, 3, 3, 3, 3, 3
(373, 1046)	5	5, 5, 5, 5, 5, 5, 5, 5
(419, 962)	1,5	1, 1, 5, 5, 1, 1, 5, 5
(431, 922)	1,5	1, 1, 1, 5, 1, 1, 5, 5

Numerical experiments

Datasets

- Dataset **F** [Fernandez and Puerto 2003]
  - 28 instances
  - source: collection obtained in pasting two mono-obj UFLP
  - size : n=30; m=90
  - correlation of objectives : ≈ 0.0
  - ranges : $c_{ij}^k \in [0, 100]$; $r_j^k \in [200, 28000]$
- Dataset **H** [Harris et al 2011]
  - 5 instances
  - source : green logistic (obj1 : cost; obj2 : CO_2)
  - size : n=10; m=2000...10000
  - correlation of objectives : ≈ 0.99
  - ranges : $c_{ij}^k \in [0, 43000]$; r_j^k unique $\forall j \in J$ per instance
- Available on the vOptLib :
  - `https://github.com/vOptSolver/vOptLib`

Computational environment

MacBook Pro, Apple M2 Pro; 32 Gb RAM

macOS 14.6.1

- our code:
  `julia` Ver 1.12.5
- MILP solver:
  `Gurobi Optimizer` Ver 13.0
- Fernandez and Puerto (2003) code:
  `Vax Fortran`, unaltered, kindly provided by the authors and executed on the same machine

Code: `https://github.com/xgandibleux/biUFLP` – Datasets: `vOptLib`

Numerical experiments (2026) — dataset F

fnames	t_1 (s)	t_2 (s)	#BPav	#BRef	t_3 (s)	$\#Y_N$	t_{123} (s)	t_{eps} (s)	t_{FP} (s)
F50-51	0.1655	0.0878	152	10	0.0357	1229	0.2890	270.1916	N.A.
F50-52	0.0075	0.0226	11	5	0.0116	408	0.0417	103.6391	44.9917
F50-53	0.1124	0.0185	21	7	0.0099	771	0.1408	162.9394	47.8149
F50-54	0.0078	0.0160	16	13	0.0120	700	0.0358	175.1433	N.A.
F50-55	0.0075	0.0112	13	9	0.0393	513	0.0580	110.5132	62.7686
F50-56	0.0082	0.0206	23	17	0.1381	729	0.1669	331.3211	N.A.
F50-57	0.0070	0.0173	17	14	0.0107	616	0.0350	240.9657	N.A.
F51-52	0.0107	0.1218	38	15	0.0322	635	0.1647	159.1519	N.A.
F51-53	0.0118	0.1229	32	9	0.0429	1047	0.1776	249.3033	50.7325
F51-54	0.0108	0.0219	24	19	0.1500	1013	0.1827	273.2615	N.A.
F51-55	0.0147	0.0253	31	22	0.1324	1111	0.1724	427.6619	N.A.
F51-56	0.0076	0.0153	14	12	0.0104	755	0.0333	159.0428	N.A.
F51-57	0.0076	0.1081	18	12	0.0363	796	0.1520	227.1591	N.A.
F52-53	0.0017	0.0047	14	8	0.0203	435	0.0267	103.5365	23.7430
F52-54	0.0006	0.0009	8	8	0.1208	47	0.1223	3.3449	45.4852
F52-55	0.0006	0.0005	3	3	0.0458	20	0.0469	2.9087	19.8469
F52-56	0.0006	0.0005	6	6	0.0091	15	0.0102	1.8619	36.7843
F52-57	0.0006	0.0009	6	6	0.1174	16	0.1189	1.5085	N.A.
F53-54	0.0009	0.0011	7	7	0.0118	333	0.0138	24.5483	32.7435
F53-55	0.0013	0.0010	6	6	0.0273	306	0.0296	69.1338	24.3891
F53-56	0.0009	0.0010	5	4	0.0118	318	0.0137	37.5853	37.7964
F53-57	0.0011	0.0005	9	9	0.1203	173	0.1219	31.3258	45.3409
F54-55	0.0003	0.0003	5	5	0.0145	37	0.0151	4.1285	N.A.
F54-56	0.0002	0.0004	4	4	0.0414	22	0.0420	3.7008	18.7721
F54-57	0.0002	0.0002	8	8	0.0097	20	0.0101	2.2026	39.9407
F55-56	0.0003	0.0000	5	5	0.1420	5	0.1423	0.2877	23.3804
F55-57	0.0002	0.0000	4	4	0.0080	4	0.0082	0.1792	19.3553
F56-57	0.0001	0.0000	3	3	0.0114	3	0.0115	0.1564	26.1427

t_1: paving – t_2: refinement – t_3: labeling – t_{123}: total (3 phases) – t_{eps}: ϵ-constraint+Gurobi – t_{FP}: Fernandez-Puerto (2003). N.A.: numerical issue on their code.

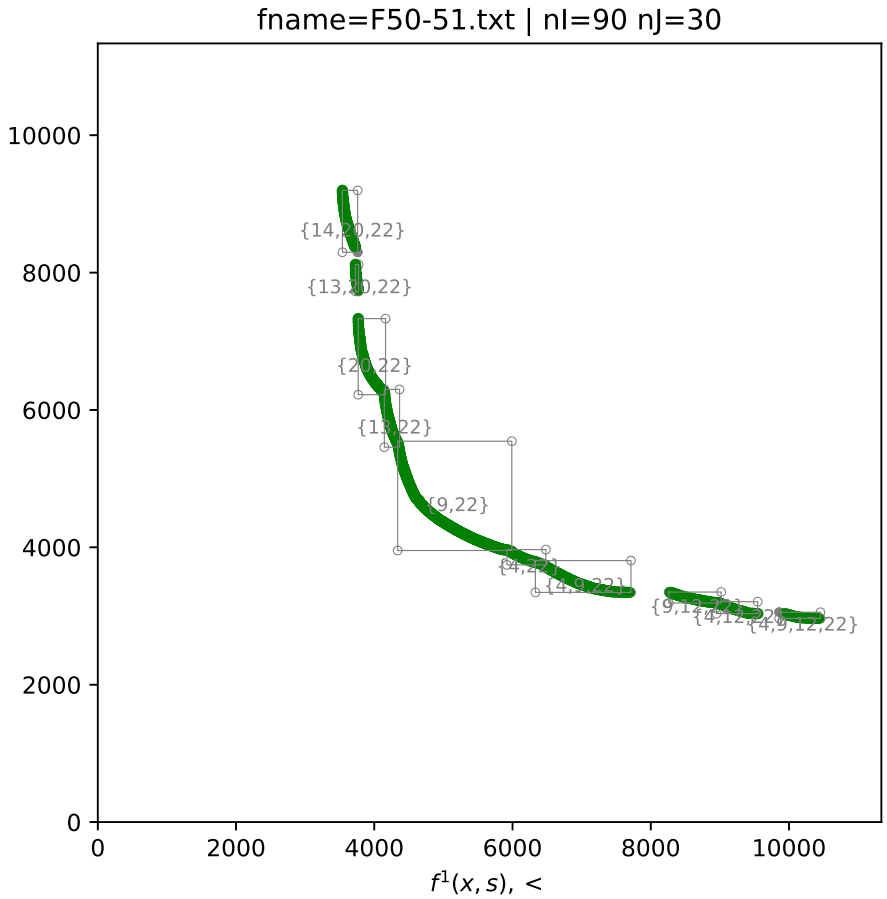
Numerical experiments (2026) — dataset H

fnames	t_1 (s)	t_2 (s)	#BPav	#BRef	t_3 (s)	$\#Y_N$	t_{123} (s)	t_{eps} (s)	t_{FP} (s)
H10-2000	0.0406	0.1922	11	11	0.0755	13412	0.3083	T.O.	N.A.
H10-4000	0.0794	0.4665	11	11	0.4475	69167	0.9934	T.O.	N.A.
H10-6000	0.1064	0.7579	13	13	1.1836	172278	2.0479	T.O.	N.A.
H10-8000	0.1495	1.3330	12	12	16.0485	731385	17.5310	T.O.	N.A.
H10-10000	0.1831	1.7360	12	12	7.9293	4931	9.8484	T.O.	N.A.

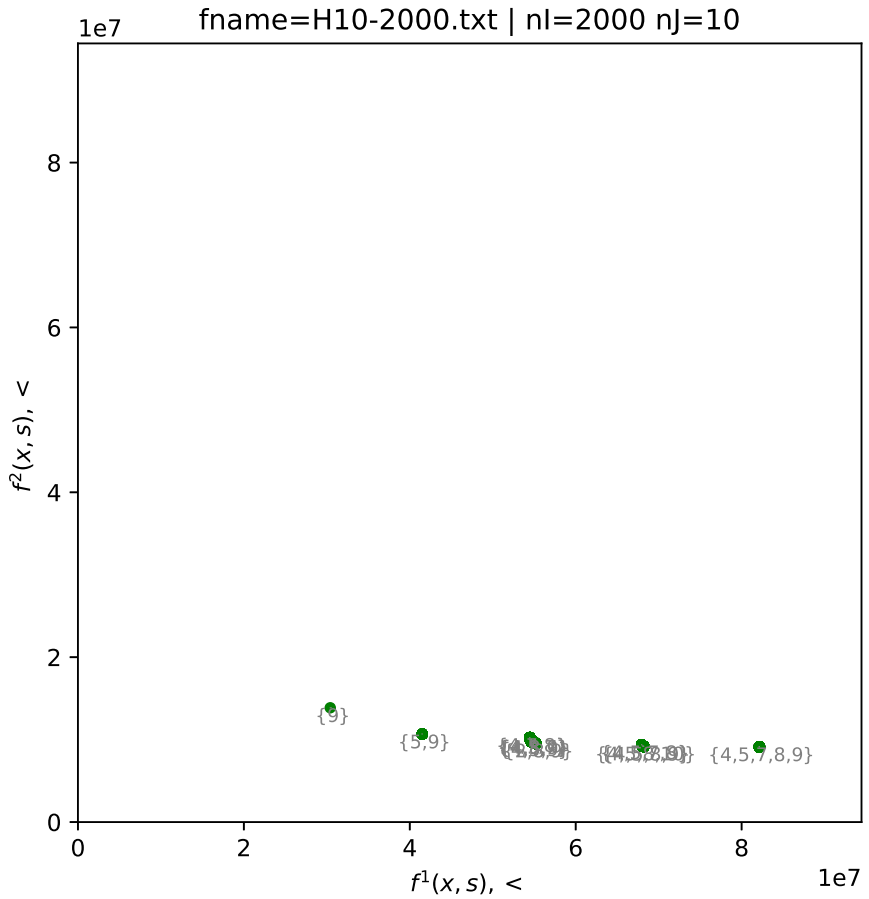
t_1: paving – t_2: refinement – t_3: labeling – t_{123}: total (3 phases) – t_{eps}: ϵ-constraint+Gurobi – t_{FP}: Fernandez-Puerto (2003).

- T.O.: ϵ-constraint+Gurobi times out (600s) on **every** H instance
- N.A.: Fernandez-Puerto (2003) cannot handle instances of this size
- Our algorithm solves all 5 instances, up to **731 385** nondominated points, in under 18s

Numerical experiments: objective space

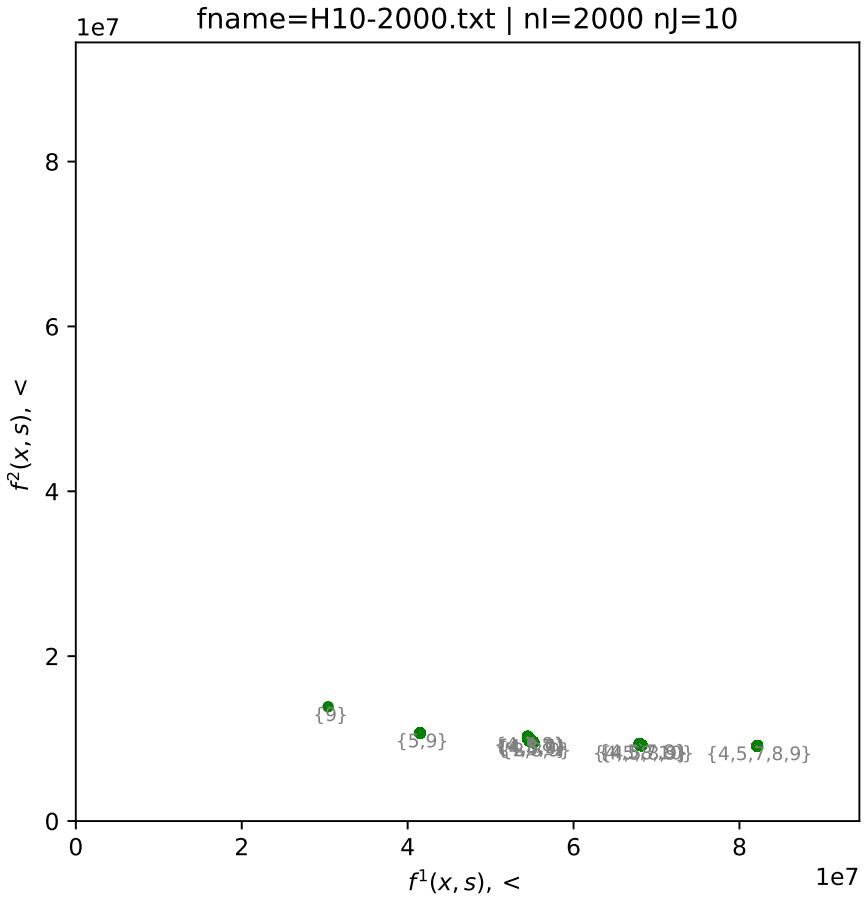


Instance F50-51

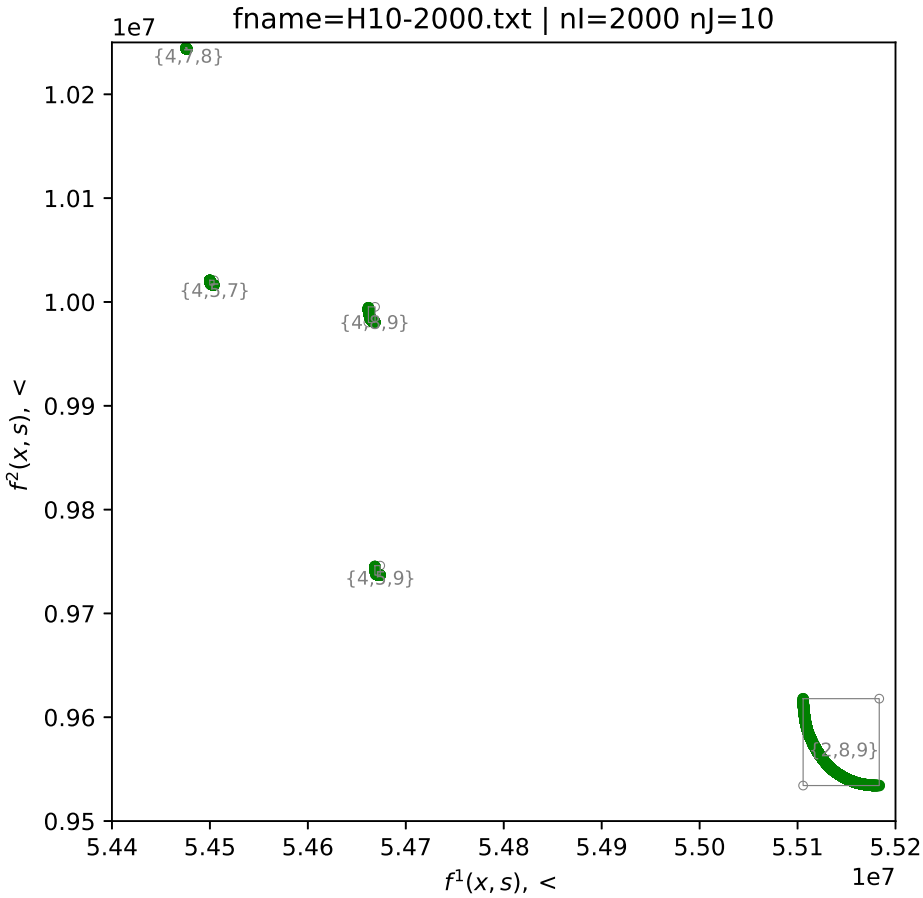


H10-2000

Numerical experiments: objective space

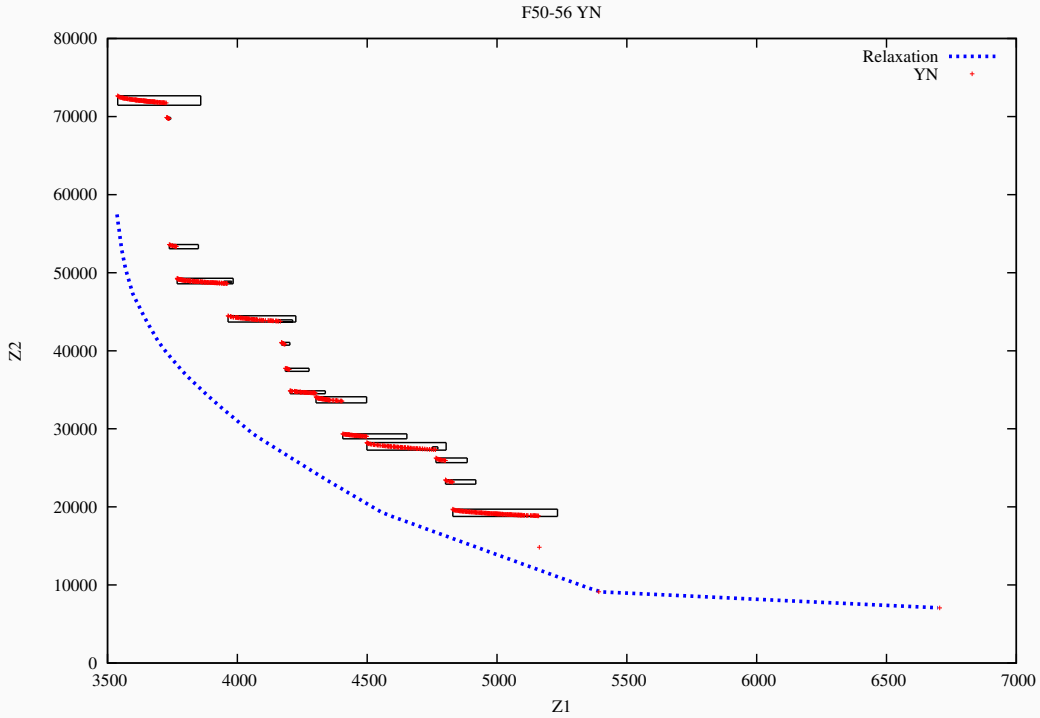


H10-2000



H10-2000 zoom

Numerical experiments: quality of the linear relaxation



- Instance F50-56: the linear relaxation (dotted) is a poor lower bound set here
- An enumerative algorithm relying on the LP relaxation would be ineffective on this instance

A new, harder dataset (2026) — dataset B

Instances

- Source: OR-Library (Beasley, 1988) — sets capa/capb
- 2 single obj. instances (100 sites $\times$ 1 000 clients), paired as (f^1, f^2)
- 10 truncated instances built ($n_I \times n_J$, from 90$\times$30 to 200$\times$50)

Results

fnames	t_1 (s)	t_2 (s)	#BPav	#BRef	t_3 (s)	$\#Y_N$	t_{123} (s)	t_{eps} (s)
Capa-capb-90-30	0.1046	0.000000	5	5	0.000600	5	0.105200	1.22
Capa-capb-90-40	0.3004	0.000000	6	6	0.000400	6	0.300800	1.96
Capa-capb-90-50	0.2430	0.000000	3	3	0.000300	3	0.243300	0.59
Capa-capb-100-30	0.0618	0.001000	7	5	0.000300	5	0.063100	1.46
Capa-capb-100-40	0.2523	0.002200	8	6	0.000700	106	0.255200	44.73
Capa-capb-100-50	1.1362	0.002800	6	6	0.001400	105	1.140400	61.00
Capa-capb-150-40	4.6705	0.575900	57	9	0.010600	1169	5.257000	T.O.
Capa-capb-150-50	20.6955	4.607000	67	10	0.013000	1326	25.315500	T.O.
Capa-capb-200-40	51.4852	11.999200	217	16	0.044000	3303	63.528400	T.O.
Capa-capb-200-50	473.3593	2.550800	182	13	0.714500	3394	476.624600	T.O.

t_1: paving – t_2: refinement – t_3: labeling – t_{123}: total (3 phases) – t_{eps}: ϵ-constraint+Gurobi

Unlike F/H: phase 1 (paving) now dominates runtime

Discussion: does a paving provide a representation? (1/2)

Sayın (2000), three quality attributes for a discrete representation:

- **coverage** (no true ND point too far from its nearest representative)
- **uniformity** (retained points evenly spaced among themselves)
- **cardinality** (a manageable number of points)

Remarks:

1. the paving (boxes) is not point-based. The candidate representation is the point set collected along the way:

$$R = \Big(\underbrace{\bigcup_{\mathcal{B}\in\mathcal{P}} \{y^{12}(\mathcal{B}), y^{21}(\mathcal{B})\}}_{U\text{, phase 1}} \cup \underbrace{\bigcup_{\mathcal{B}} S^{\mathcal{B}}}_{\text{supported, phase 2}} \Big)_N$$

2. Points can still be dominated once phase 3 completes.

Discussion: does a paving provide a representation? (2/2)

✗ **Cardinality.** $|R| = O(\#BoxPh2)$: each box contributes a bounded number of points (1 if $|J_1|=1$; else 2 corners plus a parameter-controlled number of dichotomic points) — independent of $|Y_N|$. F51–53: 9 boxes vs. $\#Y_N=1047$; H10-8000: 12 boxes vs. $\#Y_N=731\,385$.

✗ **Coverage.** Every $y \in Y_N$ lies in some box; R holds its corners, so the gap from y to R is bounded by the box's extent $(y^N - y^I)$. The reduction operators (phase 2) actively shrink this quantity (refinement is coverage-error reduction, for free).

✗ **Uniformity.** Supported points within a box are generated by a (truncated) dichotomic construction method: at each step it splits the current largest gap between adjacent supported points (a greedy gap-equalizing rule, a real mechanism in favor of local uniformity).
But without inter-box control: equalization occurs locally within each box; there is no coordination of density across box boundaries.

The Paving as a Representation of Nondominated Points for the Bi-objective Uncapacitated Facility Location Problem

Xavier GANDIBLEUX
in collaboration with Anthony PRZYBYLSKI
Nantes Université, France

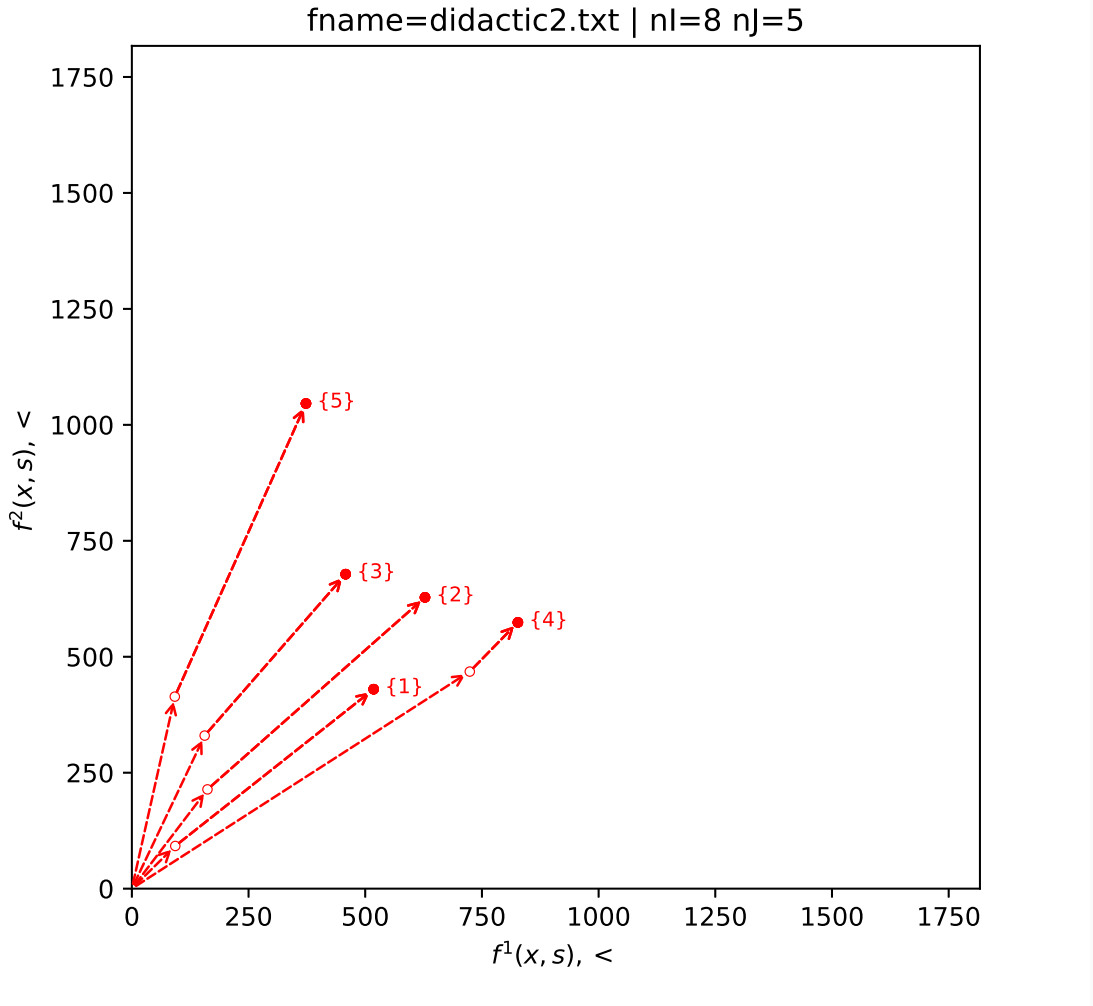
Code: `https://github.com/xgandibleux/biUFLP`

Datasets: `https://github.com/vOptSolver/vOptLib`

Didactic example (with #J=5)

Cases with only 1 facility is opened : $J_1 = \begin{cases} \{1\} \\ \{2\} \\ \{3\} \\ \{4\} \\ \{5\} \end{cases}$

1. y^R of $J_1 = \{4\}$ is dominated by e.g. y^{12} of $J_1 = \{1\}$
   ↳ test 1 ⇒ $\{4\}$ pruned
2. y^I of $J_1 = \{2\}$ is dominated by e.g. y^{12} of $J_1 = \{1\}$
   ↳ test 2 & 3 ⇒ $\{2\}$ pruned



Didactic example (with #J=5)

Cases with only 2 facilities are

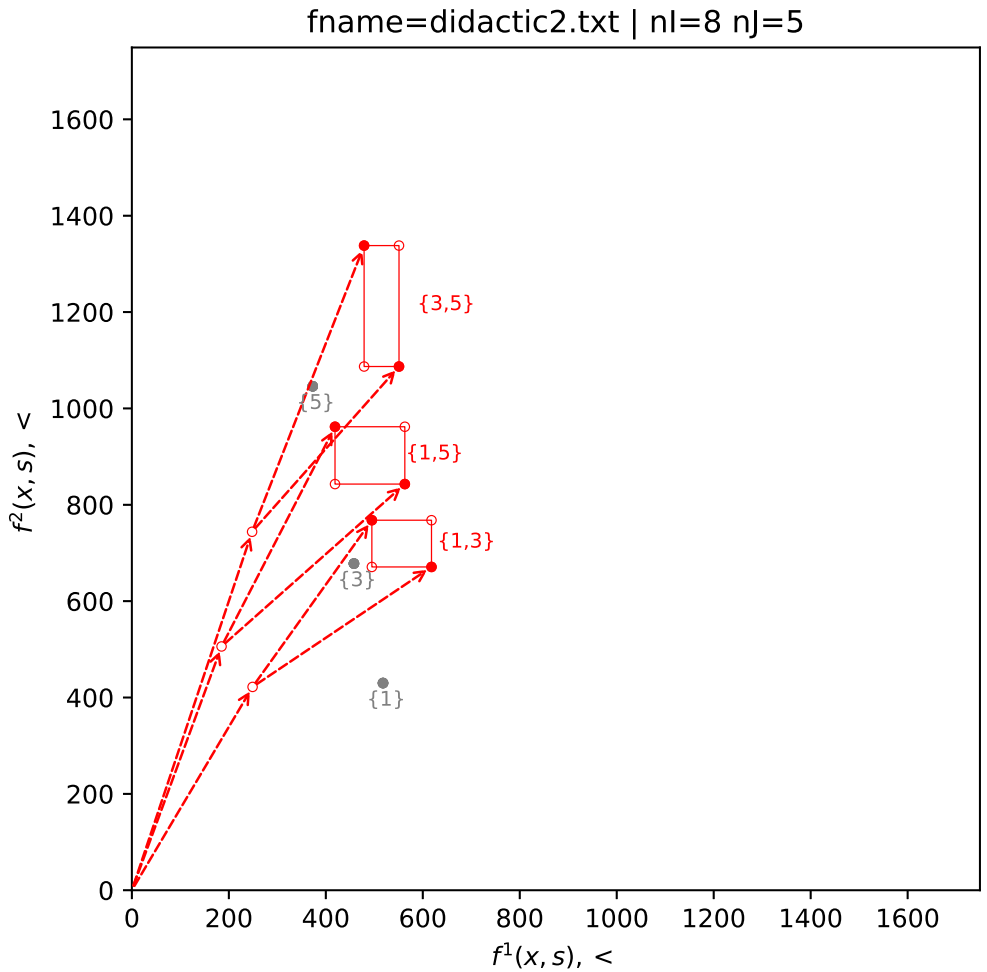
opened : $J_1 = \begin{cases} \{1,3\} \\ \{1,5\} \\ \{3,5\} \end{cases}$

Expansion of domain for the opened facilities giving 3 boxes.

1. y^l of $J_1 = \{3,5\}$ is dominated by y^{12} of $J_1 = \{1,5\}$
   ↳ test 2 & 3 $\Rightarrow$ $\{3,5\}$ pruned

Current paving at this step :

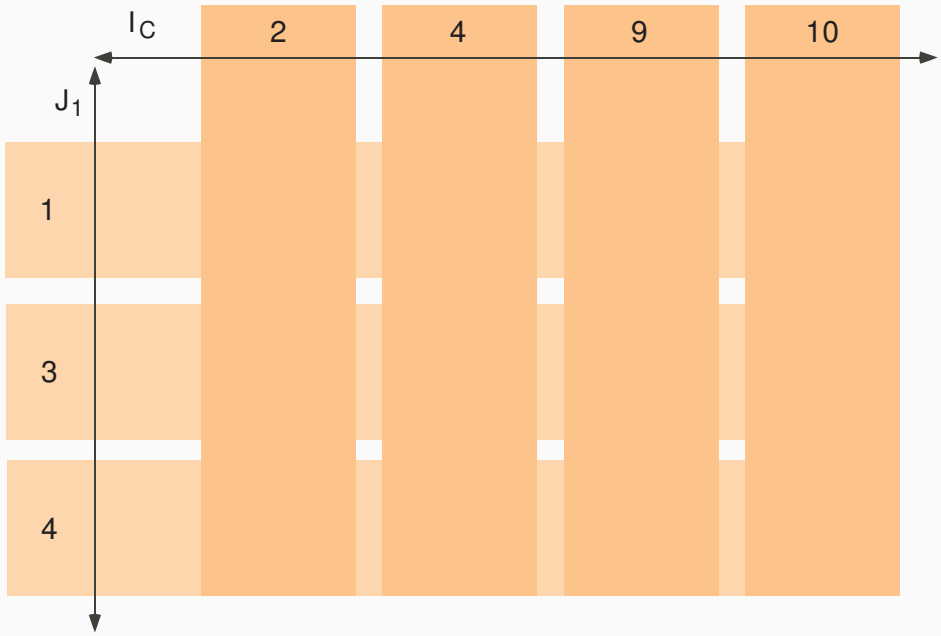
$$\Big\{\{1\},\{3\},\{5\},\{1,3\},\{1,5\}\Big\}$$



Remark: the box defined by $\{1,5\}$ may be refined

Algorithm: generation principle

- A box is defined by the opened facilities:
  e.g. $J_1 = \{1, 3, 4\}$
- Set of indexes for the non trivial assignments:
  e.g. $I_c = \{2, 4, 9, 10\}$



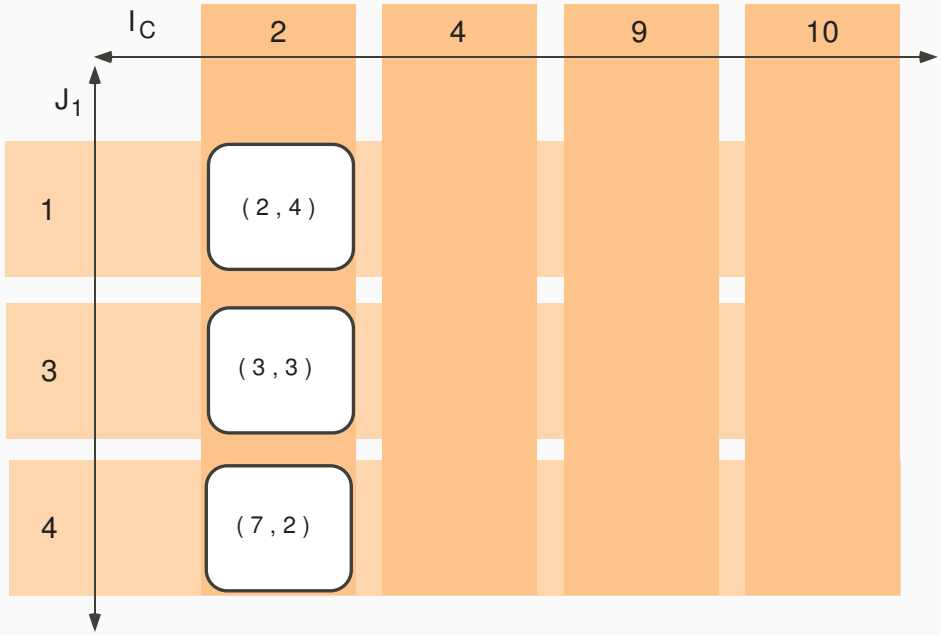
Algorithm: a label setting

Example: Ernesto Q.V. Martins. On a multicriteria shortest path problem. *European Journal of Operational Research*, 16:236–245, 1984.

Algorithm: generation principle

Customer 2 has three potential assignments:

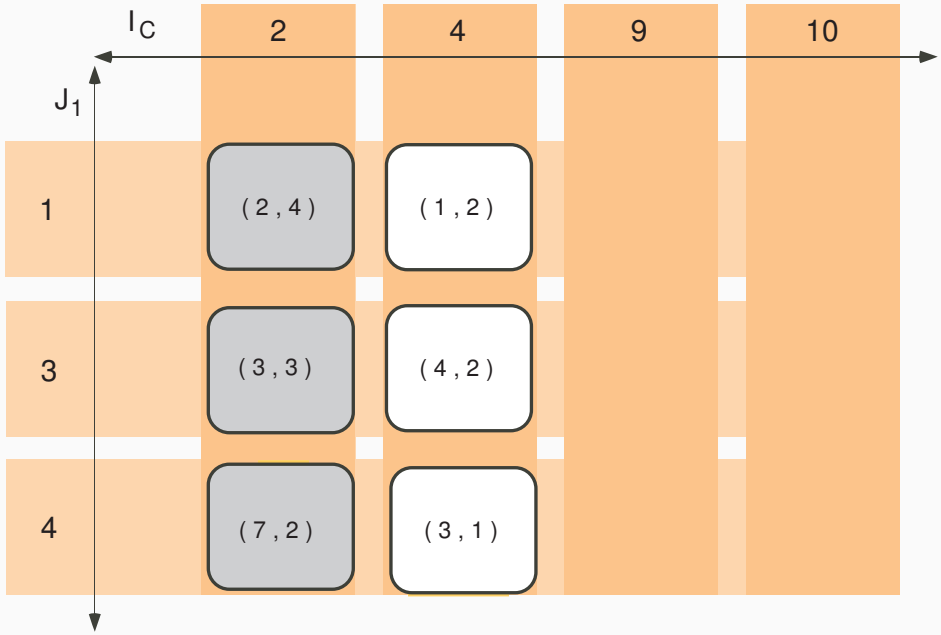
- [2, 4]
- [3, 3]
- [7, 2]



Algorithm: generation principle

Assignment of customer 4 to facility 3 is dominated.
$\Longrightarrow$ pruned

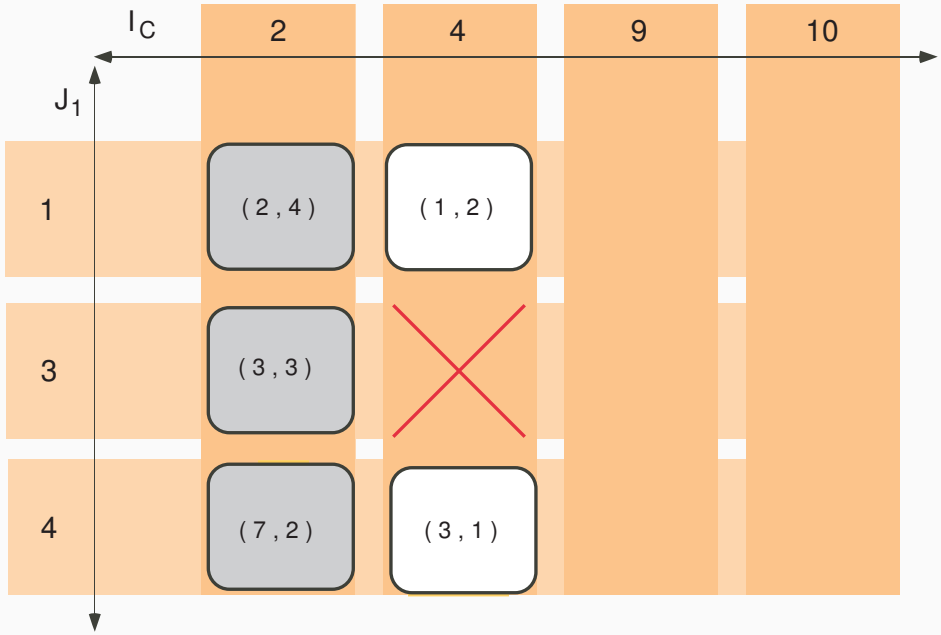
- [2, 4]
- [3, 3]
- [7, 2]



Algorithm: generation principle

For each solution already computed, the possible assignments for customer 4 are added.

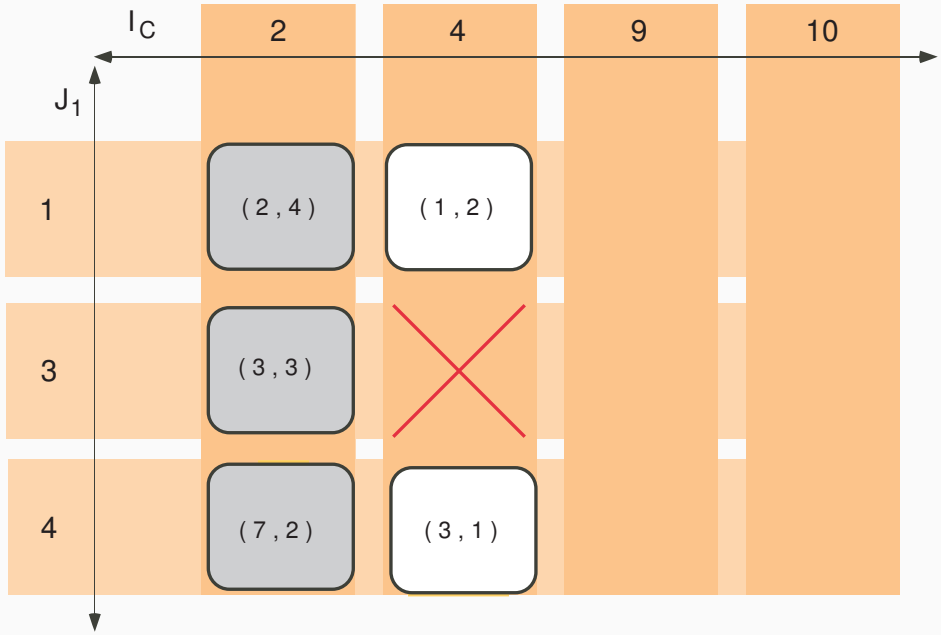
- [2, 4]
- [3, 3]
- [7, 2]



Algorithm: generation principle

For each solution already computed, the possible assignments for customer 4 are added.

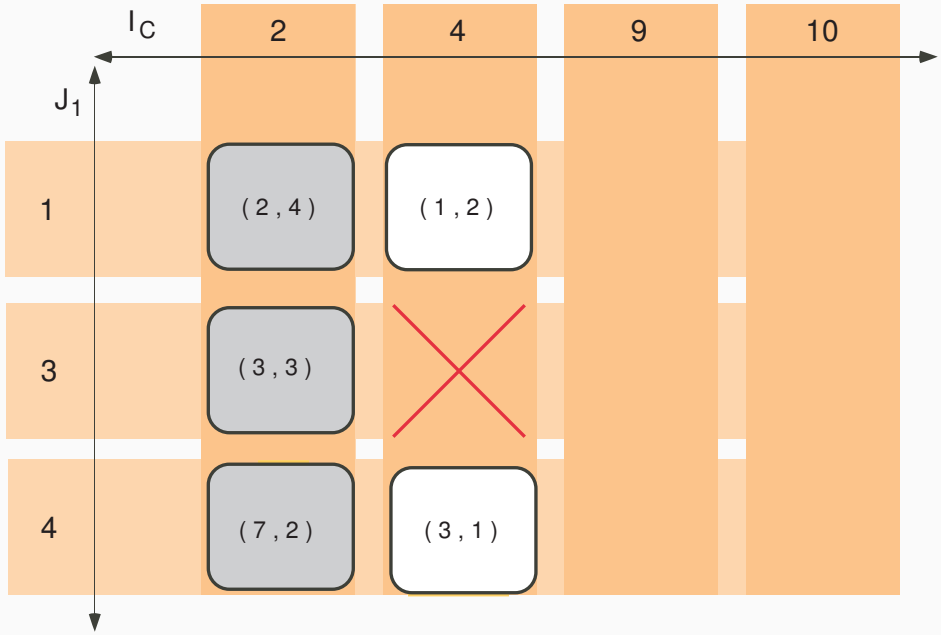
- [3, 6]
- [4, 5]
- [8, 4]
- [5, 5]
- [6, 4]
- [10, 3]



Algorithm: generation principle

For each solution already computed, the possible assignments for customer 4 are added.

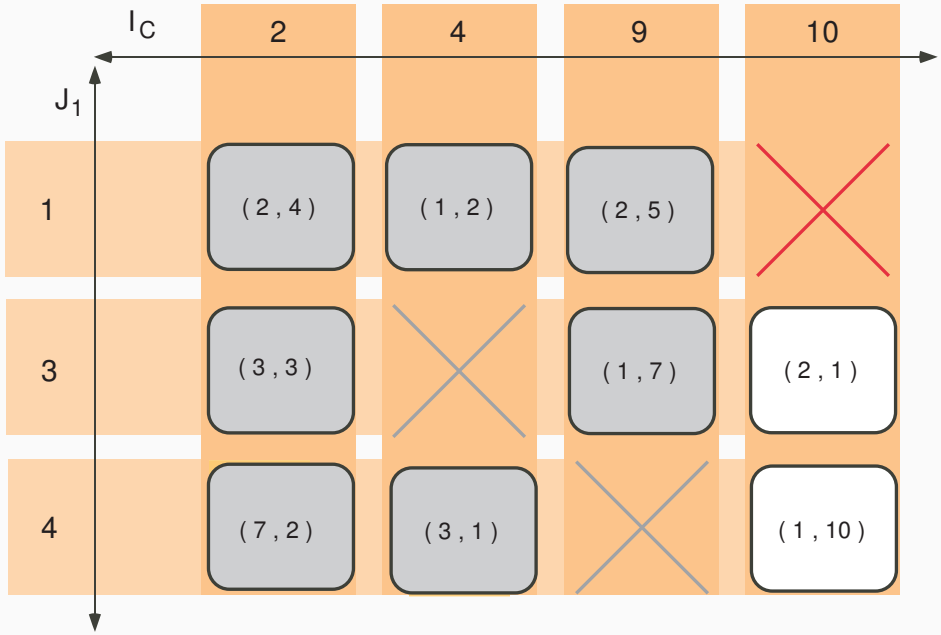
- [3, 6]
- [4, 5]
- ~~[8, 4]~~
- ~~[5, 5]~~
- [6, 4]
- [10, 3]



Algorithm: generation principle

In fine: 6 labels (solutions) not dominated.

- [7, 12]
- [8, 11]
- [11, 10]
- [14, 9]
- [6, 14]
- [5, 23]



Why is computing a supported point trivial here?

Inside a box, J_1 is **fixed**: which facilities to open — the hard, combinatorial part of 2-UFLP — is already decided.

What remains is the **allocation problem** only: assign each customer $i \in I$ to a facility $j \in J_1$. Since 2-UFLP is **uncapacitated**, there is no constraint linking customers together (no capacity, no shared flow).

For a fixed weight λ, the scalarized allocation problem therefore **decomposes into m independent, one-line problems**:

$$\min_{j \in J_1} \left(\lambda\, c_{ij}^1 + (1-\lambda)\, c_{ij}^2\right) \qquad \text{for each customer } i \in I, \text{ separately}$$

- No MIP solver, no combinatorial search: just a minimum over $|J_1|$ values, per customer
- Cost: $O(m \cdot |J_1|)$ per point — same principle already used for y^{12}, y^{21} (lexicographic optima)

$\Rightarrow$ Phase 2 can afford to compute several supported points cheaply to feed the reduction tests